

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

题 目： 无线通信系统链路速率建模

摘 要：

无线通信技术当前已实现普及化应用，MIMO-OFDM（多输入多输出-正交频分复用）与 TxBF（发送波束赋形）技术打破信道容量瓶颈；在实际应用中通过物理层抽象对信道的速率进行估计常常很有必要，但大规模系统中信道呈现多维度耦合的特征，传统速率估计方法存在等效 SINR 匹配度低及 TxBF 开关机制差异等局限，导致估计偏差大，引发资源错配、链路误码率升高甚至高可靠业务中断等问题。针对此，本文根据现网采集的 MIMO-OFDM 系统数据集，通过数据预处理优化、模型改进与基于物理结构感知的多模态特征融合，围绕三种核心场景实现传输速率精准估计。

对于问题一：针对关闭 TxBF 场景下 EESM 模型参数拟合难、数据时间不同步、mcs 非整数离散分布的难点，本文首先采用 IQR（四分位距）方法剔除信道状态信息（CSI）与传输速率采集时间差异异常值，确保数据时间同步；随后通过统计 mcs 离散值的最小差值划分映射区间，并将区间线性归一化至 $[0,1]$ 以适配 EESM 模型输出函数值域；构建 **L2 范数损失函数**并引入物理约束惩罚项，结合 **Adam 自适应优化算法**，通过融合一阶动量，即平滑更新方向与二阶动量，动态调整学习率，实现了等效映射系数 α 与 SINR 缩放系数 β 的高效收敛，拟合了 EESM 的等效映射系数 α 与 SINR 缩放系数 β ，最终完成数据集 B 的载波单元 SINR 计算、等效 SINR 映射与速率估计，三个子数据集验证集准确率分别达 **70.97%、66.94%、61.94%**。

对于问题二：针对关闭 TxBF 场景下传统 EESM 对于低频子载波 SINR 特征效果差、忽略 MIMO 信道矩阵结构与 SINR 统计规律、映射函数近似误差大的难点，本文进一步用 IQR 方法剔除噪声异常值提升数据纯度；基于符号错误传输概率理论，**修正 Chernoff 边界近似**，提出**改进型指数映射函数**，降低宽 SNR 区间的近似误差；引入 MIMO 信道奇异值特征与 SINR 统计特征扩展特征维度，通过网格搜索确定目标函数正则化项系数最优取值，精准平衡拟合精度与过拟合风险；模型验证集准确率提升至 **79.03%、75.97%、71.94%**，较问题一平均提升 **9.03%**。

对于问题三：针对开启 TxBF 场景下信道结构变化、子载波增益差异显著、传统模型适配性差的难点，本文基于主奇异值占比，提出适配波束赋形场景的**加权改进型指数映射**等效 SINR 模型；基于 MIMO 物理结构感知，构建 CSI 与 SINR 多模态特征融合的信道 Transformer（MFF-CT）模型：将等效信干噪比 $SINR_{eff}$ 的标量特征融合 CSI 数据，提取特征序列并标准化，结合 Positional Encoding 保留其原有**频率顺序信息**，利用“波束赋形”

的空间相关性**优化模型注意力权重**，最后经平均池化与分类头输出 mcs 预测，有效适配 TxBF 的空间域信号，显著提升开启 TxBF 场景下的速率估计精度，模型验证集准确率提升至 **87.5%**，较问题二有了最大 **15.56%**的提升。

综上所述，本文通过分场景优化数据预处理策略、改进传统模型映射机制、构建基于 MIMO 物理结构感知的多模态特征融合 MFF-CT Transformer 模型，逐步解决了 MIMO-OFDM 的速率估计难题，模型准确率逐场景提升。研究成果不仅为无线通信系统的资源调度提供可靠依据，减少业务重传能耗与中断损失，更可为 5G 及后续通信系统中远程医疗、工业控制等高带宽低时延业务的稳定运行提供技术支撑，助力无线通信系统向高效化、可靠化方向发展。

关键词： MIMO-OFDM；等效 SINR；EESM 模型；Adam 优化；MFF-CT Transformer 模型；速率估计

目 录

目 录	3
1. 研究背景及问题重述.....	4
1.1 研究背景.....	4
1.2 问题重述.....	4
1.3 研究思路.....	4
2. 模型假设与符号说明.....	6
2.1 模型假设.....	6
2.2 符号说明.....	6
3. 问题一：EESM 模型参数拟合与关闭 TxBF 场景下的速率估计	8
3.1 问题分析.....	8
3.2 数据预处理.....	8
3.3 模型构建与求解.....	10
3.4 结果分析.....	14
3.5 问题小结.....	19
4. 问题二：关闭 TxBF 场景下的新型速率估计模型构建与速率估计	20
4.1 问题分析.....	20
4.2 数据预处理.....	20
4.3 模型构建与求解.....	21
4.4 结果分析.....	23
4.5 问题小结.....	26
5. 问题三：开启 TxBF 场景下的新型速率估计模型构建与速率估计	27
5.1 问题分析.....	27
5.2 数据预处理.....	27
5.3 模型构建与求解.....	28
5.4 结果分析.....	33
5.4.1 SINR 计算	33
5.4.2 等效 SINR 计算	33
5.4.3 mcs 预测	33
5.4.4 模型评估.....	34
5.5 问题小结.....	37
6. 模型评价	38
6.1 模型优点.....	38
6.2 模型缺点.....	38
7. 结论与展望	39
7.1 研究结论.....	39
7.2 未来展望.....	39
参考文献	40
附录 A: matlab 代码.....	41
附录 B: Python 代码.....	58

1. 研究背景及问题重述

1.1 研究背景

无线通信技术已完成从早期无线电通信到 Wi-Fi 6、5G 商用普及的跨越式发展，传输速率实现从 kbps 级到 Gbps 级的突破。这一演进的核心驱动力源于关键技术的融合创新：OFDM（正交频分复用）与 MIMO（多输入多输出）技术的结合打破了信道容量瓶颈，而 TxBF（发送波束赋形）技术的补充增强，进一步提升了信号传输的定向增益与抗干扰能力。当前，大规模 MIMO-OFDM 系统已广泛部署，天线数量与载波单元规模显著扩大，使得无线信道呈现动态性强、多维度耦合的特征，对实际传输速率的精准估计需求愈发迫切。

实际传输速率是衡量链路性能的核心指标，受 MIMO 信道矩阵特性、载波单元 SINR、噪声干扰、调制编码方式等影响，与理论香农容量存在显著差异。传统速率估计方法（如 EESM 模型）存在明显局限：一是 SINR 等效映射依赖经验参数拟合，未充分考虑信道矩阵动态变化与载波差异，等效 SINR 匹配度低；二是忽略底噪时变特性及 TxBF 开关的信道机制差异，导致估计值与实测速率偏差大。这种偏差会引发无线资源错配、链路误码率升高，极端时甚至中断高可靠性业务（如远程医疗、工业控制），影响通信服务质量。

精准的速率估计具有重要价值：一方面可减少业务重传能耗与中断损失，保障高带宽低时延业务稳定运行；另一方面能为资源调度提供可靠依据，降低运维成本、延长设备寿命，最终实现无线通信系统高效、可靠运行。因此，针对大规模 MIMO-OFDM 系统，及开启 TxBF 的场景，优化现有的速率估计模型以提升准确性，已成为降低链路风险、实现系统高效可靠运行的必要途径。

1.2 问题重述

基于以上背景本题目提供了现网环境下采集的无线通信数据集，要求基于 MIMO-OFDM 系统的数学原理，解决以下三个核心问题，实现对实际传输速率的精准估计：

问题一：EESM 模型参数拟合与关闭 TxBF 场景下的速率估计

基于完备数据集 A（含关闭 TxBF 的 MIMO 信道矩阵与对应传输速率等信息），拟合 EESM 模型的关键参数等效映射系数 α 与 SINR 缩放系数 β ；针对不完备数据集 B（不含传输速率）完成以下任务：

1. 载波单元 SINR 计算：计算不完备数据集 B 每个载波单元的信干噪比
2. SINR 等效映射：基于完备数据集 A 拟合得到的 EESM 模型参数完成 B 中多载波 SINR 的降维，得到等效 SINR。
3. 建模信道传输速率 $rate$ ，完成 B 中传输速率预测。

问题二：关闭 TxBF 场景下的新型速率估计模型构建与速率估计

在关闭 TxBF 场景下，以完备数据集 A 为基础，构建新型速率估计模型，利用该模型对不完备数据集 B 进行传输速率估计。

问题三：开启 TxBF 场景下的新型速率估计模型构建与速率估计

针对开启 TxBF 场景，基于完备数据集 A（含开启 TxBF 的 MIMO 信道矩阵与对应传输速率标签等信息），适配 TxBF 技术带来的信道结构变化与信号传输机制差异，构建适用于该场景的新型速率估计模型，利用该模型对不完备数据集 B（不含传输速率）进行传输速率 $rate_{txbf}$ 估计。

1.3 研究思路

全文求解思路如图 1。

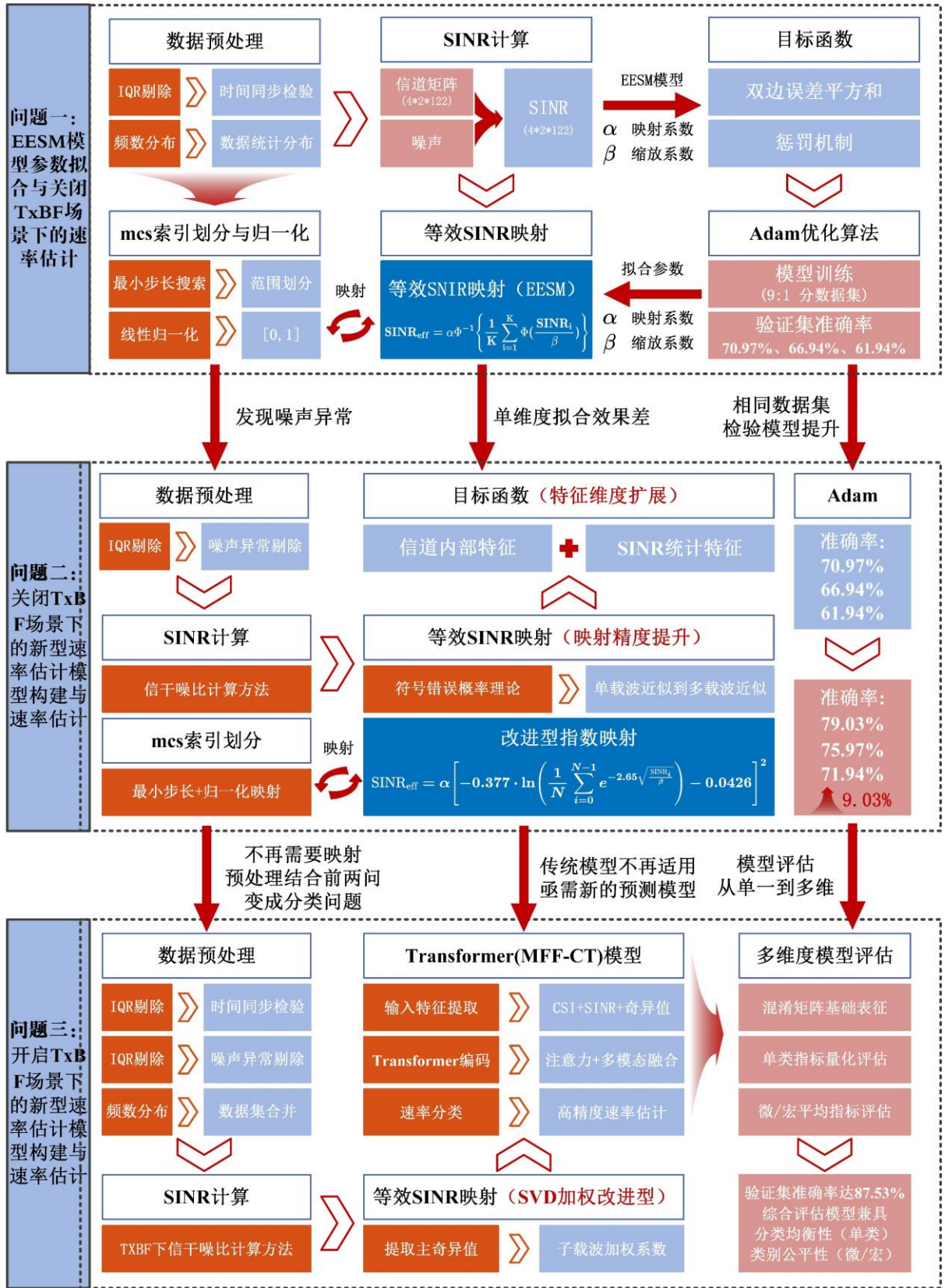


图 1 全文求解思路图

2. 模型假设与符号说明

2.1 模型假设

在本文的建模过程中，在不影响模型意义与输出精度的前提下，为使得模型简单、明确，本文做出如下假设：

1. 设备与环境稳定性假设：

数据采集期间，无线基站与终端硬件性能稳定，且终端严格遵循“移动 - 静止”预设流程，采集环境无突发强干扰，确保信道与速率数据反映真实通信链路特性。

2. 载波单元独立性假设：

OFDM 系统中 122 个载波单元满足正交性，无跨载波干扰，每个载波单元的信道系数仅表征该载波的传输特性，与其他载波无关，确保各载波 SINR 可独立计算。

3. 特征一致性假设：

MIMO 信道的奇异值统计特征、SINR 统计特征与传输速率的关联关系，在数据集 A 与 B 中保持一致，无场景迁移导致的特征失效。

4. mcs 与实际速率的映射波动有界假设

同一 mcs 索引对应的实际传输速率存在波动，但波动范围应小于 mcs 分辨率，且波动服从零均值高斯分布，不改变 mcs 越大、实际速率越高的单调关系，模型预测时可采用 mcs 对应的速率区间替代单一速率值。

2.2 符号说明

题目所给建模有关变量与数据集有关变量如表 1。

表 1 变量符号及其说明

序号	符号	符号说明
1	$SINR_i$	载波单元 <i>i</i> 上信干噪比
2	$SINR_{eff}$	等效信噪比
3	mcs	传输速率
4	H_i	信道矩阵
5	α	等效映射系数
6	β	缩放系数
7	l_k	mcs 划分步长
8	$lower^k$	第 <i>k</i> 个样本的 <i>mcs</i> 下限
9	$upper^k$	第 <i>k</i> 个样本的 <i>mcs</i> 上限
10	J_{min}	目标函数
11	λ	约束违背惩罚系数
12	γ	正则化系数
13	w_k	第 <i>k</i> 个特征的特征权重
14	η	学习率
15	m_t	t 时刻一阶动量
16	v_t	t 时刻二阶动量
17	ϵ	数值稳定性常数
18	β_k	k 阶动量衰减系数
19	w_k	第 <i>k</i> 个特征的特征权重
20	$\sigma_{i,j}$	第 <i>i</i> 个载波单元的第 <i>j</i> 个奇异值

序号	符号	符号说明
21	$\bar{\sigma}_i$	第 <i>i</i> 个载波单元的奇异值均值
22	$\overline{SINR}$	SINR 均值
23	σ_{SINR}^2	SINR 方差
24	S_{SINR}^2	SINR 偏度
26	U_i	第 <i>i</i> 个载波单元信道矩阵的左酉矩阵
27	V_i	第 <i>i</i> 个载波单元信道矩阵的波束赋形权值矩阵
28	D_i	第 <i>i</i> 个载波单元信道矩阵的奇异值对角矩阵
29	y_i	第 <i>i</i> 个载波单元信道矩阵的接收信号向量
30	s_i	第 <i>i</i> 个载波单元的发送信号向量
31	w_i	第 <i>k</i> 个发送天线的权值向量
32	L	MIMO 信道可传输信号数量
33	$rate_{txbf}$	开启 TxBF 的实际传输速率
34	$csi_matrix_ri_ci$	信道状态信息矩阵切分列，ri 代表接受天线，ci 代表发射天线
35	dfx_time	传输速率采集时间
36	csi_time	信道状态信息采集时间
37	$beamforming_en$	波束赋型开启指示
38	p_i	第 <i>i</i> 个子载波的加权系数

3. 问题一：EESM 模型参数拟合与关闭 TxBF 场景下的速率估计

3.1 问题分析

问题一核心任务围绕 MIMO-OFDM 系统（关闭 TxBF）的链路速率建模展开，可以拆分为模型参数拟合与未知数据速率估计两阶段完成。针对模型参数拟合，问题一要求利用完备数据集 A 中数据拟合 EESM 模型中的关键参数 α 与 β 。针对未知速率阶段，本题要求对不完备数据集 B 依次计算每个载波单元的信干噪比（SINR），对所有载波单元的 SINR 进行等效映射，基于拟合后的 EESM 模型，建模并输出信道传输速率 rate。

针对问题一，本文求解思路如图 2。

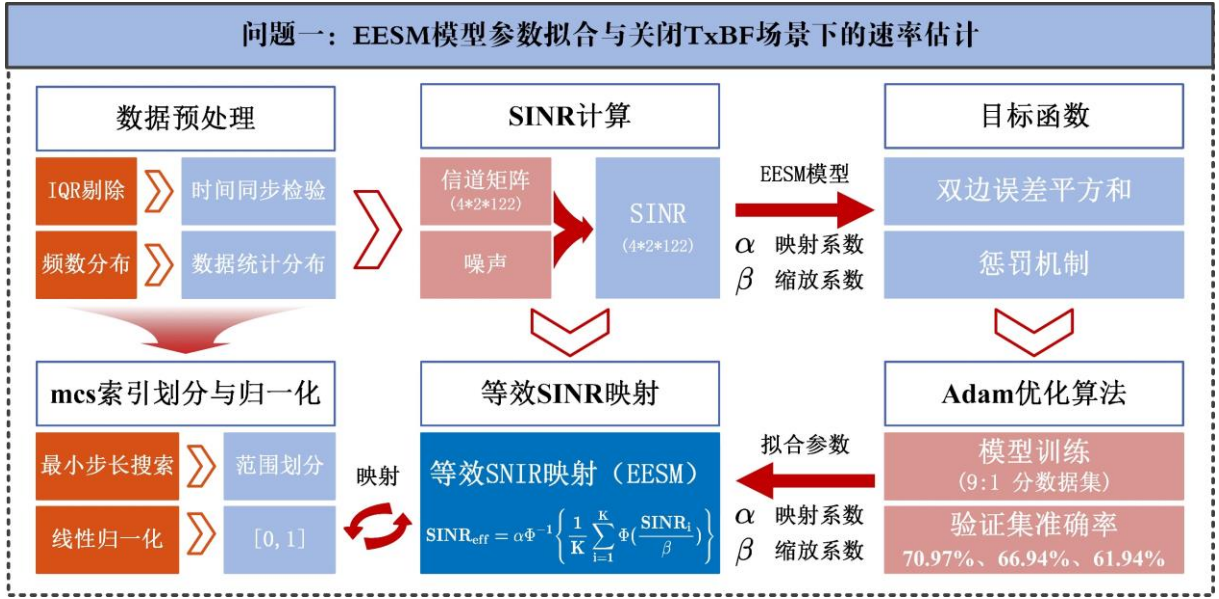


图 2 问题一求解思路

首先对题目中提供的数据集进行预处理，通过计算信道状态信息采集时间与传输速率采集时间的差值，采用 IQR 方法剔除时间异常值，确保数据在时间上的匹配性；随后基于 MIMO-OFDM 系统原理和关闭 TxBF 条件下的 SINR 计算公式，计算每个载波单元的 SINR 值；针对数据集中非整数 mcs 值的特点，设计区间划分与归一化策略，将 mcs 映射到 [0,1] 区间；构建包含上下边界约束和惩罚机制的目标函数，采用 Adam 自适应优化算法求解 EESM 模型的关键参数 α 与 β ；利用优化后的模型对各载波单元 SINR 进行等效映射，得到等效 SINR；最后将等效 SINR 映射为归一化速率，并反归一化为原始 mcs 空间，输出速率索引估计结果，同时使用验证集评估模型性能。

3.2 数据预处理

3.2.1 数据采集时间同步检验

首先对于题目所给原始数据进行预处理，题目给定没有波束赋形的数据中一共有三个终端 1、2、3，其 MAC 值分别为 341c-f0d4-70be、bc98-2983-907b、d0c1-bfe4-18f0，即三个子数据集，每一个子数据集都有 A 数据（包含传输速率 mcs ）和 B 数据（未包含传输速率 mcs ），首先将数据进行读取与检验，结果发现传输速率采集时间 dfx_time 与信道状态信息采集时间 csi_time 并不完全相同，而是存在一定差值，即采集时间差如图 3。

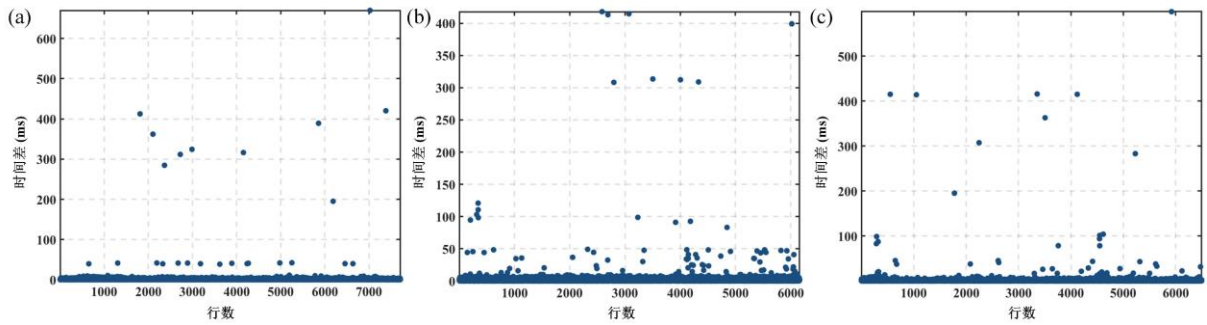


图 3 (a)-(c)三个数据集的时间差分布图

由图 2 可以看出，三个终端的数据均存在不同程度的采集时间偏差，时间差主要集中在 0~5 ms 范围内，但也有部分数据点的时间差明显超出这一区间，达到甚至超过 10 ms。这表明在数据采集过程中，信道状态信息与传输速率信息并非严格同步，可能受到终端移动、网络延迟、设备采样频率差异等因素影响。时间差的存在可能导致在后续建模中使用不匹配的信道与速率数据，从而影响模型的准确性。从散点图可以进一步观察到，时间差随数据行号的变化趋势并不明显，说明采集时间偏差具有随机性，不存在明显的时间漂移现象。这一特性使得基于阈值的异常值剔除方法成为可行的预处理手段。

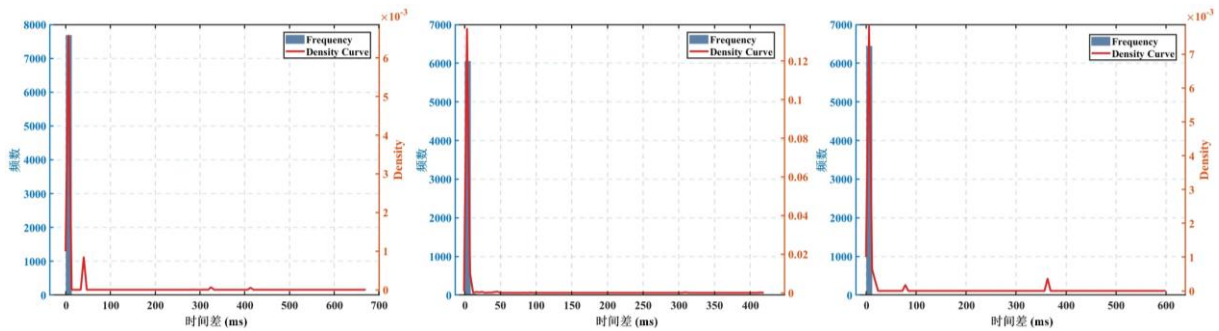


图 4 (a)-(c)三个数据集的时间差分布直方图与密度曲线

图 3(a)-(c)分别展示了三个终端数据集在时间差分布上的直方图与核密度估计曲线。从直方图的分布形态来看，时间差主要集中在 0~5 ms 区间，这与图 2 的观察结果一致。密度曲线进一步显示，时间差分布在峰值附近较为集中，随着偏离，概率密度迅速下降，但在较大时间差处仍有少量分布，说明存在少量时间同步偏差较大的异常点。通过对比三个子图可以发现，不同终端的时间差分布在峰值位置和宽度上存在细微差异，这可能与各终端的硬件采样特性、移动状态及网络环境有关。

综上，为了提高数据质量，本文采用 IQR（四分位距）方法对时间差进行异常值检测，IQR 作为一种基于数据分布四分位数的稳健异常值识别方法，核心是通过数据的离散程度划定合理范围，避免受极端值影响。利用 IQR 方法对每个数据集单独计算，计算每个数据集的 IQR 阈值，将超过阈值的时间差视为异常值并剔除。处理结果如表 2 所示。

表 2 数据预处理前后大小

终端 MAC 值	原始数据大小（行）	IQR 阈值（ms）	预处理后大小（行）
341c-f0d4-70be	7712	0,4.472000	7475
bc98-2983-907b	6161	0,7.566500	6022
d0c1-bfe4-18f0	6496	0,4.185000	6197

最终，通过剔除时间差异异常的数据点，我们获得了更为同步的信道状态信息与传输速

率数据对，这有利于提高后续 EESM 模型参数拟合的准确性，减少因时间不匹配引入的建模误差，为建立高精度的链路速率预测模型奠定了基础。

3.3 模型构建与求解

3.3.1 目标函数建立

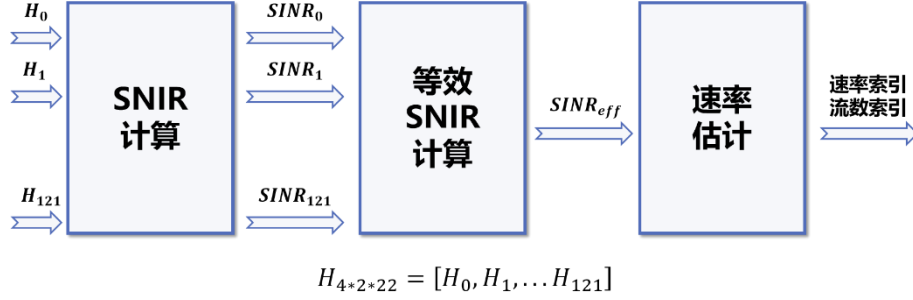


图 5 实际传输速率估计方法

对于 EESM 模型拟合的问题，本文首先按照实际传输速率估计方法进行分析，对于 MIMO-OFDM 系统中信道矩阵 $H_{4 \times 2 \times 22}$ ，首先按照关闭 TxBF 情况下 SINR 计算公式计算：

$$SINR_i = \|H_i\|^2 / \|n_i\|^2 \quad (1)$$

由于直接对 $SINR_i$ 进行加权处理存在技术局限，业界普遍采用等效 $SINR$ 映射 $SINR_{eff}$ 的方式进行降维操作。在理想信道条件下，OFDM 系统中的无线信道传输速率可以由 $SINR_i$ 进行表示：

$$I(SINR_{eff}) = \sum_i I(SINR_{eff}) P_i \quad (2)$$

其中 I 为度量函数， P_i 为概率质量函数，当度量函数为指数信息度量时，即：

$$I_{EXP}(\gamma) = 1 - \exp(-\gamma) \quad (3)$$

此时该 $SINR$ 映射为 EESM(exponential ESM)模型，当指数型 ESM 可以推广到一般情形时，其等效 $SINR$ 可用以下公式表示：

$$SINR_{eff} = \alpha \Phi^{-1} \left\{ \frac{1}{K} \sum_{i=1}^K \Phi(SINR_i / \beta) \right\} \quad (4)$$

式中 Φ 为映射函数，对(4)式两边进行 Φ 映射，得到式(5)如下：

$$\Phi(SINR_{eff}) = \alpha \frac{1}{K} \sum_{i=1}^K \Phi(SINR_i / \beta) = rate \quad (5)$$

由此可知，我们建立起了 EESM 模型下的传输速率 $rate$ 与 $SINR_i$ 的关系，考虑到题目仅仅给出 mcs 即传输速率索引，本文首先对数据的 mcs 分布特征进行统计分析：

分析结果如图 6 所示，数据集中的 mcs 呈现离散分布特性，多为非整数，且不同子集的 mcs 取值范围存在显著差异，推断原因是无线信道质量会收到信号强度随距离波动、多径效应致信号衰落、同频/邻道干扰及噪声电平变化，使信道质量发生动态变化，当信道条件处于两个标准 MCS 等级的边界时，设备可能会选择一个中间值来平衡性能和稳定性，即实际速率出现非整数的情况。

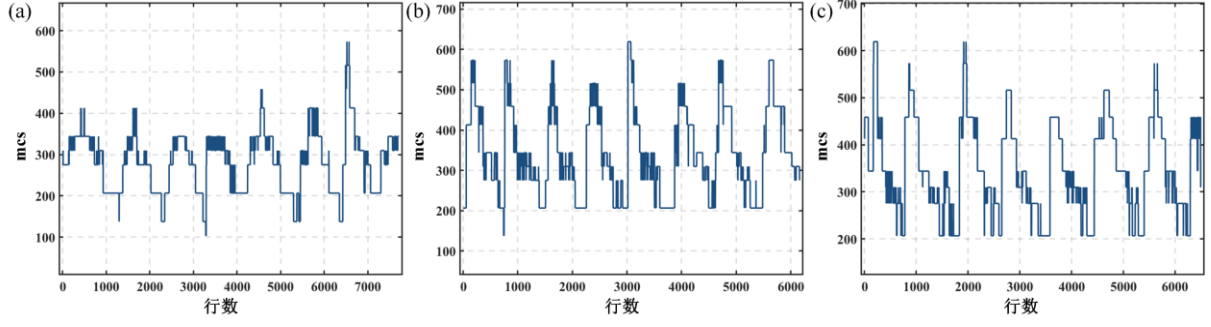


图 6 数据集中 mcs 分布图

根据图 6 中 mcs 分布情况, EESM 模型预测的 $rate$ 与 mcs 呈非线性关系, 而最小二乘法 (OLS) 隐含假设是目标与特征呈近似线性关系, 直接应用 OLS 会导致模型无法准确捕捉这种非线性映射。同时, 对于 EESM 模型, 其映射函数为指数函数, 其计算得到的 $rate$ 与 $SINR_{eff}$ 的关系如图 7, 从图中可以发现按照 EESM 公式预测得到的 $rate \in [0,1]$, 这一特性意味着不能对直接对原始 mcs 数据构建目标函数进行拟合。

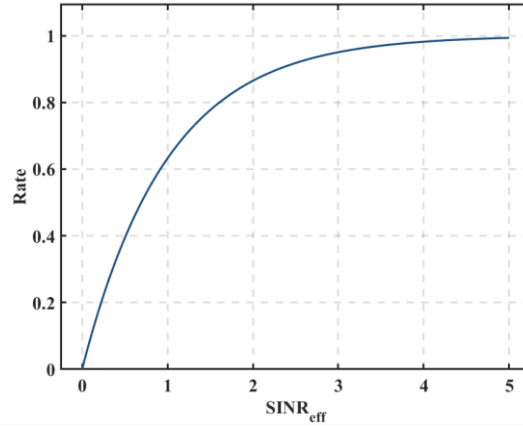


图 7 EESM 模型中 $rate$ 与 $SINR_{eff}$ 的关系

综上所述, 我们建立以下数学模型, 首先针对数据集内的第 k 行数据, 其对应等效信干噪比 $SINR_{eff}^k$ 与传输速率 mcs^k , 每一个 mcs^k 都对应一个实际传输速率 $rate^k$ 的区间范围:

$$rate^k \in [lower^k, upper^k] \quad (6)$$

为实现模型参数的最优估计, 建立以下带约束的目标函数:

$$J_{min} = \sum_{k=1}^{rows} (\Phi(SINR_{eff}^k) - upper^k)^2 + \sum_{k=1}^{rows} (\Phi(SINR_{eff}^k) - lower^k)^2 + \lambda \cdot P \quad (7)$$

其中 λ 为惩罚系数, P 为约束违背惩罚项, 其取值规则如下:

$$lower^k < \Phi(SINR_{eff}^k) < upper^k, P = 0 \quad (8)$$

$$\Phi(SINR_{eff}^k) > upper^k \text{ or } \Phi(SINR_{eff}^k) < lower^k, P = 1 \quad (9)$$

本目标函数设计具有以下特点: 采用双边误差平方和形式, 同时考虑上下边界约束, 提高估计稳健性; 同时引入惩罚机制处理约束违背情况, 增强模型泛化能力。该目标函数能够有效平衡拟合精度与约束满足度, 为 EESM 模型参数估计提供了合理的优化方向。

3.3.2 速率索引划分与归一化

由上文的速率索引 mcs 分布（呈现离散分布，多为非整数）可知，无法直接将 $SINR_{eff}$ 直接映射为单一的 mcs 值上，理论 mcs 与子载波数、编码率、空间流数等因素有关，考虑到数据集都是采用 MIMO-OFDM 系统的同样载波数收集而得，因此可以采用一个统一的步长 l_k 对每一个 mcs_k 值进行划分，将其划分为下式：

$$mcs_k = (mcs_k - l_k/2, mcs_k + l_k/2) \quad (10)$$

对于步长的选取，本文对三个数据中 mcs 差值进行统计，选取其最小值作为 l_k ，经过比较，选定 $l_k =$ 。

步长选取后，我们对 mcs 进行归一化到 $[0,1]$ 上，具体地，对每一个划分后的 mcs 值 mcs'_k ，采用线性归一化方法：

$$\overline{mcs}_k = \frac{mcs'_k - mcs_{min}}{mcs_{max} - mcs_{min}} \quad (11)$$

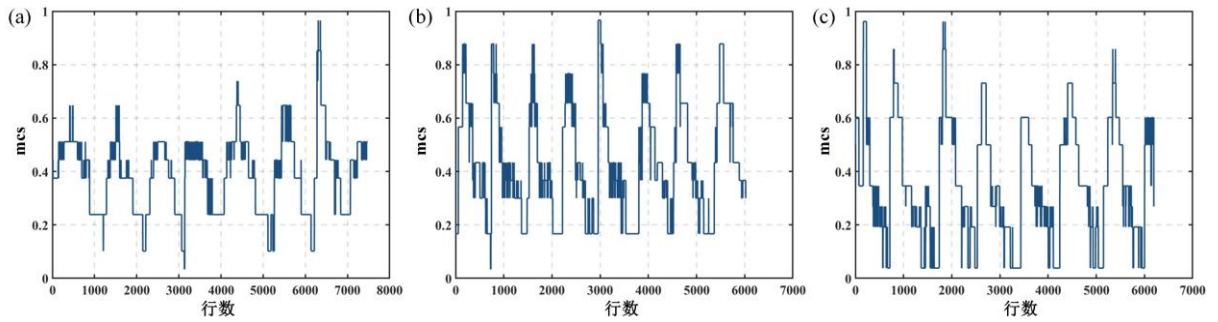


图 8 归一化后的数据分布图

图 8 给出了归一化后 mcs 数据的整体分布情况，可以看出其分布趋势与原始数据保持一致，但数值范围已被压缩到统一区间。图 9 展示了归一化后数据的直方图与核密度估计曲线，能够更直观地观察到数据的集中区域和概率密度变化趋势。由图可见，归一化后的数据主要集中在若干个峰值附近，这与 MIMO-OFDM 系统在不同信道条件下对应的调制编码方案分布特性相符。

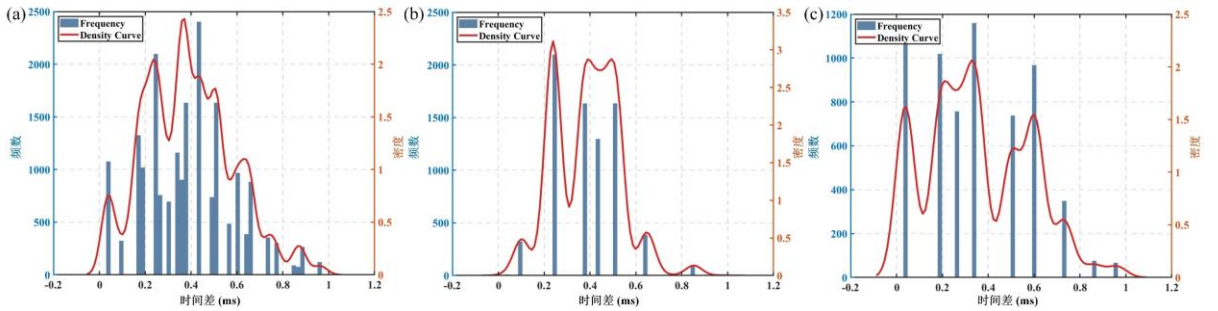


图 9 归一化后的数据分布直方图与密度曲线

这种处理方法不仅解决了非整数 mcs 值带来的映射难题，还为后续带约束的目标函数构建奠定了基础，使模型能够更准确地捕捉 SINR 与传输速率之间的非线性关系。

3.3.3 Adam 优化算法

针对 3.3.1 节构建的带约束目标函数 J_{min} ，其优化变量为 EESM 模型核心参数 α 、 β ，而惩罚系数采用人为设定方式；同时目标函数含非线性映射项 $\Phi(\cdot)$ 与约束违背惩罚项 P，整体属于非凸优化问题。针对该问题具有的小参量、大数据特点：仅有两个核心参数但样本规模庞大，目标函数非凸且含指数映射与约束惩罚项。Adam 凭借自适应学习率与动量机制，能够为 α, β 自动分配合适步长并平滑更新方向，在保证估计精度的同时提升收敛效率，尤其适合非凸优化与大规模数据场景，其偏差修正机制还可确保迭代初期的稳定性，

因此与本题的优化需求高度契合。传统梯度下降算法存在学习率固定、收敛速度慢及易陷入局部最优的缺陷，因此本文采用 Adam (Adaptive Moment Estimation) 自适应优化算法，通过融合动量法与自适应学习率机制，实现 α 、 β 的高效、稳定估计，同时结合人为设定的 λ 保障约束条件的刚性。

对于算法收敛性验证通过 Lyapunov 稳定性分析，当目标函数 J_{min} 满足 Lipschitz 连续条件时(梯度 Lyapunov 常数 $L < \infty$ ， λ 固定， $\Phi(\cdot)$ 为光滑指数函数，故满足此条件)，优化变量 $\{\theta_i\}$ 满足：

$$\lim_{t \rightarrow \infty} \mathbb{E}[\|\nabla J_{min}(\theta_t)\|^2] = 0 \quad (11)$$

即算法能收敛至 α 、 β 的平稳点。Adam 算法通过计算梯度的一阶矩（动量项）与二阶矩（自适应学习率项），为优化变量 α 、 β 动态调整更新步长。设定待优化向量如下式：

$$\theta = [\alpha, \beta]^T \quad (12)$$

算法流程如图 8，将分为以下步骤：

1) 参数初始化：

根据 EESM 模型物理意义与行业经验值，设置 $\alpha_0 = 1$ （初始等效映射系数）、 $\beta_0 = 1$ （初始 SINR 缩放系数），但其需要经过映射放缩。平衡约束惩罚强度与模型拟合灵活性，将惩罚系数设定为 0.1。

固定如下算法超参数，一阶动量 $m_0 = 0$ （初始梯度均值估计），二阶动量 $v_0 = 0$ （初始梯度方差估计），迭代次数为 $t = 0$ 、基础学习率 $\eta = 10^{-4}$ 、衰减系数 $\beta_1 = 0.9$ （平滑梯度方向、 $\beta_2 = 0.999$ （稳定学习率调整），数值稳定性常数 $\epsilon = 10^{-8}$ （避免分母为 0）。

2) 迭代更新

计算梯度：在第 t 次迭代中，仅针对 θ 进行更新，对目标函数 J_{min} 求优化值 θ_t 的梯度：

$$g_t = \nabla_{\theta} J_{min}(\theta_t) = \left[\frac{\partial J_{min}}{\partial \alpha}, \frac{\partial J_{min}}{\partial \beta} \right]^T \quad (13)$$

其中 $\frac{\partial J_{min}}{\partial \alpha}, \frac{\partial J_{min}}{\partial \beta}$ 需结合式(5)的 $\Phi(\cdot)$ 链式求导规则，以及(8) – (9)中 P 的得分特性。

更新一阶动量：一阶动量为梯度的指数移动平均，用于平滑 α 、 β 的更新方向：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (14)$$

更新二阶动量：二阶动量为梯度平方的指数移动平均，为 α 、 β 分配自适应学习率，适配两者对传输速率估计的不同敏感度：

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (15)$$

偏差修正：消除初始时刻 m_0, v_0 为 0 导致的动量估计偏差，确保迭代前期 α 、 β 更新的准确性：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (16)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (17)$$

参数更新：结合修正后的动量项，仅更新模型核心映射参数：

$$\theta_{t-1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (18)$$

迭代终止判断：参数更新后需通过最大迭代条件或损失收敛条件是否终止迭代，确保收敛性与计算效率平衡，即需满足以下式子：

$$t < t_{max} \quad (19)$$

$$|J_{min}(t+1) - J_{min}(t)| < \epsilon \quad (20)$$

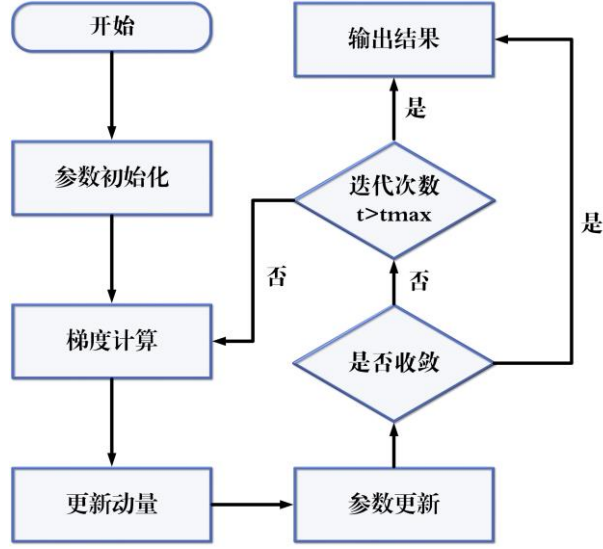


图 10 Adam 算法流程框图

综上，针对 EESM 模型 α （等效映射系数）与 β （SINR 缩放系数）的优化，本文基于含 $\Phi(\cdot)$ 与 P 的非凸目标函数 J_{min} ，提出 Adam 自适应优化方案。该方案融合一阶、二阶动量机制，解决传统梯度下降缺陷，结合 λ 与迭代终止条件，平衡约束刚性与计算效率。

3.4 结果分析

3.4.1 模型训练

在完成 3.3 节的目标函数构建与参数优化方法设计后，本节将针对三个子数据集分别进行 EESM 模型训练。之所以按子数据集独立训练，而非合并所有数据统一拟合，是因为不同终端在硬件特性、信道环境和数据统计特征上存在显著差异，同时从 3.2 节的时间差分布、 mcs 分布直方图及密度曲线可见，三个终端的采集时间偏差特性、速率分布范围与集中趋势各不相同，这意味着它们的信道质量演化规律与SINR动态范围存在本质区别。若直接合并数据，模型参数将被“拉平”以适配整体分布，导致对各终端的针对性下降，甚至产生系统性偏差；而按子数据集分别训练，则可使每个模型在自身数据的统计特性下寻找最优 α 与 β ，从而在保持物理意义合理性的同时，最大化各自的预测精度与约束满足度。

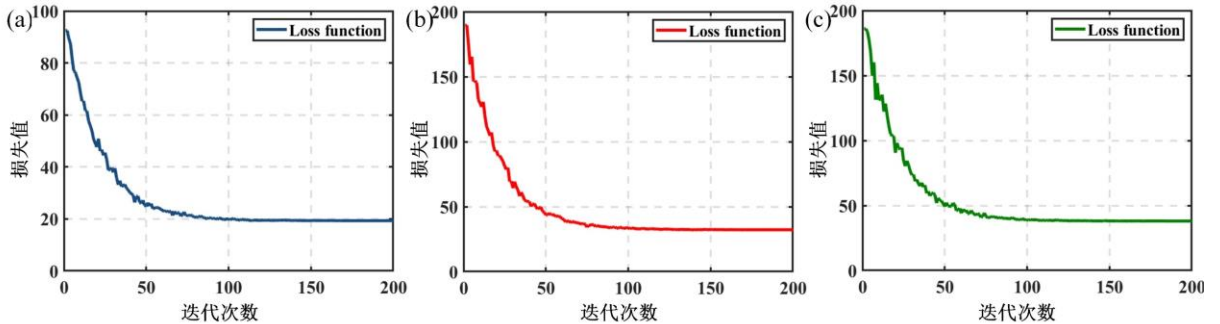


图 11 (a)-(c)三个子数据集损失函数值随迭代次数的变化曲线

图 11 展示了三个子数据集在训练过程中损失函数值随迭代次数的变化曲线。可以观察到，所有模型在训练初期均呈现快速下降趋势，在前 50 轮迭代中损失值大幅降低，随后下降速度逐渐减缓。大约在 100 轮迭代后，损失函数值的变化已不明显，到 150 轮迭代时，

三个模型的损失函数值均趋于稳定，表明此时参数已接近最优解。

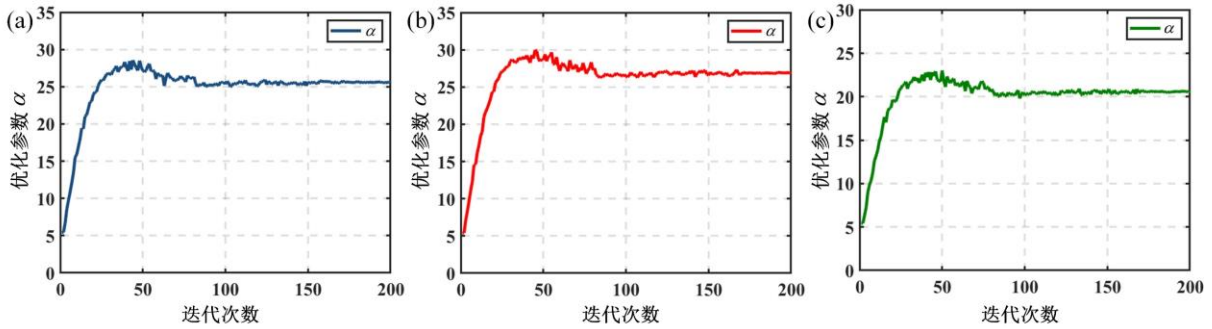


图 12 (a)-(c)三个子数据集优化参数 α 随迭代次数的变化曲线

图 12 和图 13 分别展示了优化参数 α 与 β 随迭代次数的变化曲线。可以看出，各终端的 α 与 β 参数在训练初期均有较大幅度的调整，随着迭代次数增加，变化幅度逐渐减小，在 150 轮迭代后基本收敛到稳定值。这一收敛过程验证了 Adam 优化算法的有效性，能够在合理的迭代次数内找到最优参数组合。

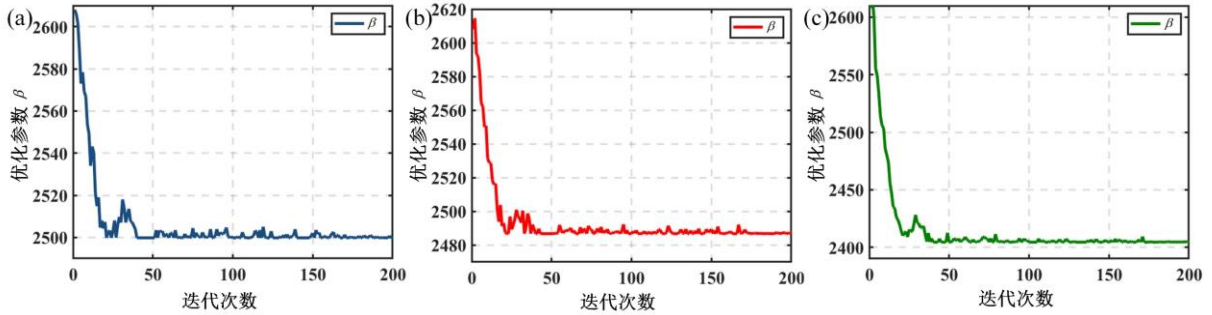


图 13 (a)-(c)三个子数据集优化参数 β 随迭代次数的变化曲线

表 3 总结了三个子数据集在 150 轮迭代后的最终训练结果。可以看出，三个模型的损失函数值存在一定差异，这反映了不同终端数据的拟合难度有所不同。MAC 地址为 341c-f0d4-70be 的终端获得了最低的损失值 96.4335，其对应的 α 与 β 分别为 25.5973 和 2499.8417。而 MAC 地址为 d0c1-bfe4-18f0 的终端损失值最高为 189.9567，其 α 值为 20.5775 也明显低于其他两个终端， β 值为 2404.4134 同样为三者中最小。bc98-2983-907b 终端的各项指标则介于两者之间。这些差异进一步验证了按子数据集分别训练的必要性。不同终端需要不同的 α 与 β 参数才能达到最佳拟合效果，这与各终端的硬件特性、信道环境以及数据统计特征密切相关。通过独立训练，每个模型都能针对特定终端的数据特点进行优化，从而获得更精准的速率预测能力。

表 3 三个子集拟合的损失函数值与优化参数

终端 MAC 值	损失函数值（150 轮迭代后）	优化参数 α	优化参数 β
341c-f0d4-70be	96.4335	25.5973	2499.8417
bc98-2983-907b	160.8806	26.8807	2499.8216
d0c1-bfe4-18f0	189.9567	20.5775	2404.4134

利用拟合后的参数对验证集（将各子集 A 中按照 9:1 进行再划分为训练集和验证集，方便对比，每一个数据集有 620 个点的验证集）进行预测，预测结果如图 14。

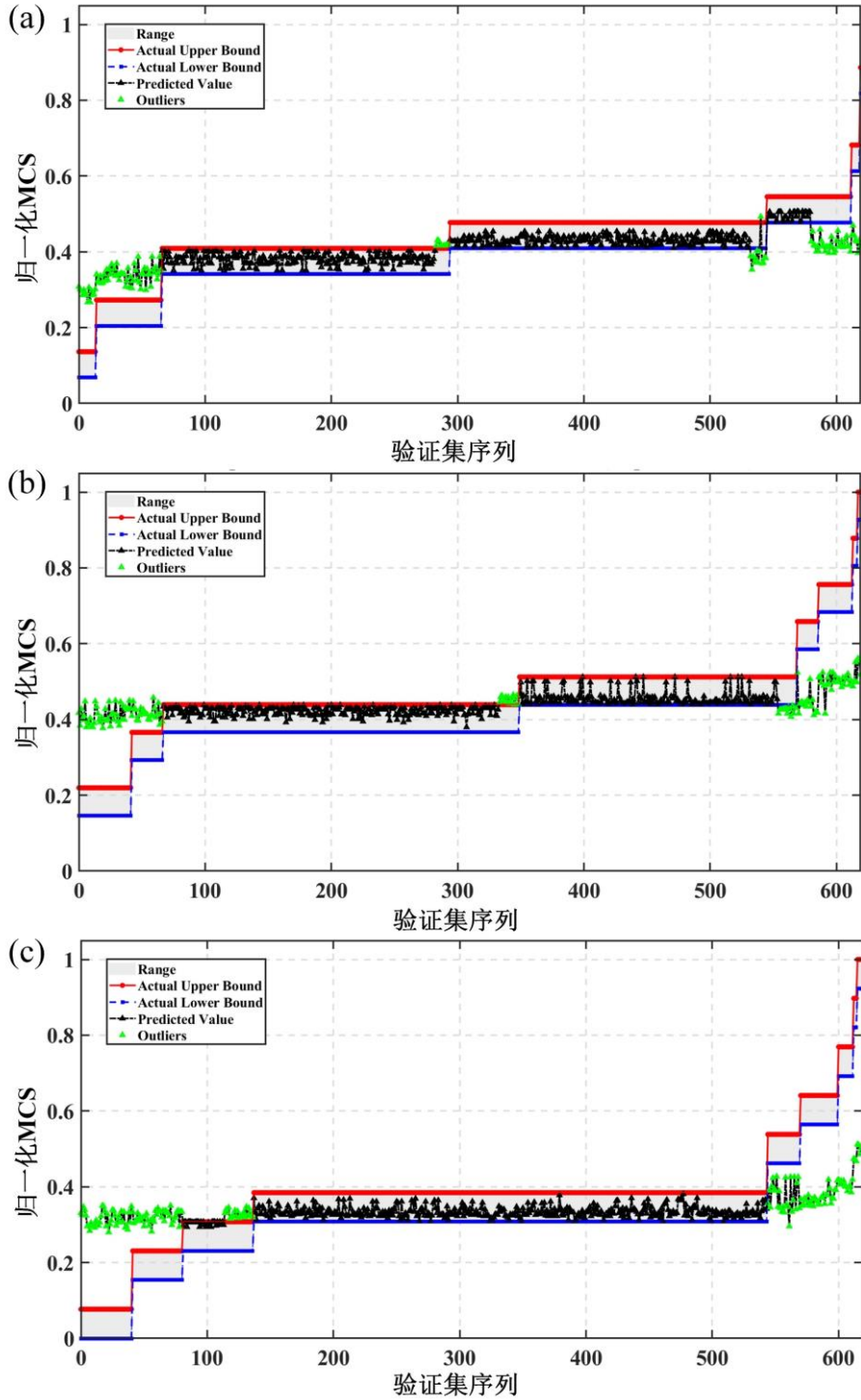


图 14 (a)-(c)三个子数据集验证集预测效果

由图 14 可知，三个子数据集上的模型对于等效 mcs 预测准确率分别为：**70.97%**、**66.94%**、**61.94%**。针对第三个数据集准确率最低的问题，结合其训练数据特征与模型学习规律，深入分析可知，该训练集中某一特定 mcs 值的样本数量过度集中，导致预测效果不佳。尽管所提方法在三个子数据集上均实现了较好的拟合与收敛，但整体预测准确率仍有提升空间，这主要源于特征维度的限制。当前模型仅依赖 SINR 作为输入特征，虽然 SINR 是反映信道质量的重要指标，但在实际通信系统中，无线链路的传输速率还受多种因素影响，特征数量少是限制目前准确率的主要原因。

总而言之，本节通过对三个子数据集的独立训练与结果分析，验证了 EESM 模型在不同终端条件下的适配性与有效性。结果表明，针对不同硬件特性与信道环境的数据分别优化参数，能够显著提升模型的预测精度与稳定性。尽管个别数据集由于样本分布不均衡导致性能略逊，但整体而言，所提方法在保持物理意义一致性的同时，展现了良好的速率预测能力，为后续的自适应传输与资源优化提供了可靠的技术支撑。

3.4.2 SINR 计算

针对问题一的第一小问，本文直接按照实际传输速率估计方法中的式(1)进行计算，对于三个子数据集集中的 B 内数据，将原始数据中的单位统一后进行计算，最终得到的结果如图 15，大部分 SINR 都在 40 左右，且频数分布曲线均呈现出“单峰对称”的形态。

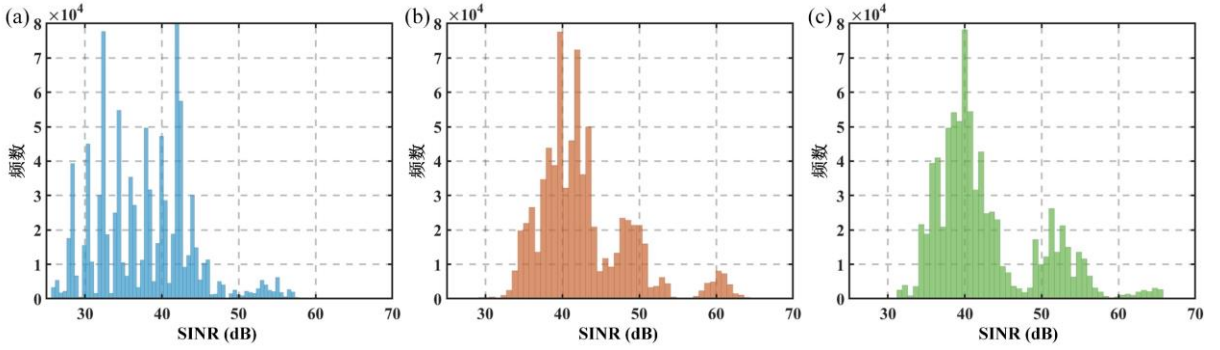


图 15 (a)-(c)三个子数据集载波单元的 SINR 频数分布图

为进一步量化 SINR 结果的稳定性，本文对三个子数据集中每行对应的 122 个载波单元的数值进行了标准差计算，如图 16，在同一测试周期内，122 个载波单元的 SINR 数值波动范围极小，信道间的信号质量差异很小，如果差异很大，可能说明该数据采集时受到干扰，失去反映真实信道特性的价值。

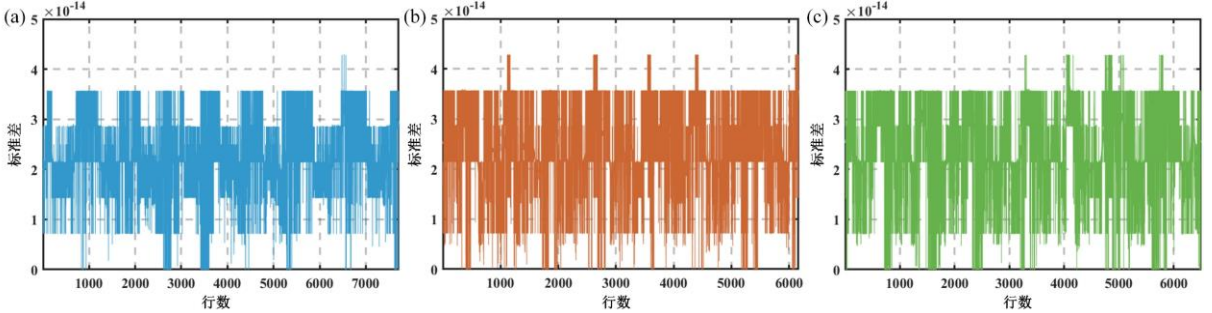


图 16 (a)-(c)三个子数据集载波单元的 SINR 标准差

3.4.3 等效 SINR 映射

针对问题一的第二小问，等效 SINR 映射是将多载波系统中各子载波上的 SINR 值转化为一个能够代表整体信道质量的单一指标，本文采用 EESM 模型进行拟合计算，将 3.4.1 节中优化参数 α 与 β 代入公式(4)，得到三个子数据集的等效 SINR 如图 17。

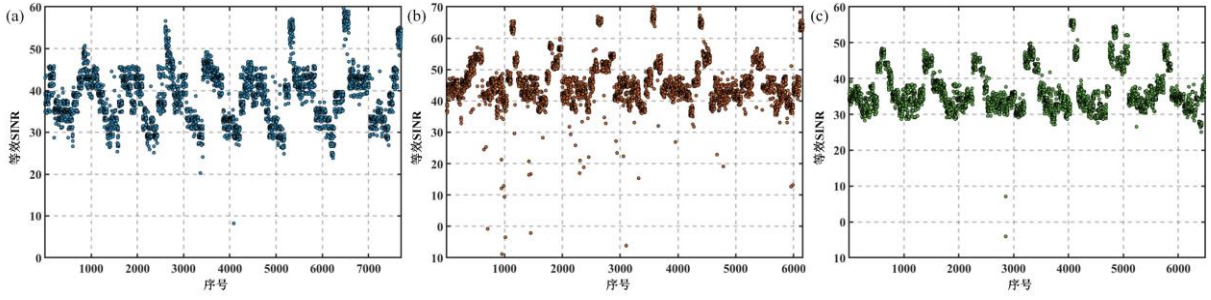


图 17 (a)-(c)三个子数据集的等效 SINR

3.4.4 传输速率预测

针对问题一的第三小问，对于信道传输速率的建模，由于 mcs 呈现非整数离散分布，无法直接将其作为连续变量与 EESM 模型输出的归一化速率建立线性映射，因此需要结合速率索引划分与归一化的处理方法，构建一套完整的从 $SINR$ 到 mcs 的非线性映射机制。首先依据速率索引划分原则，先对 A 数据集中的 mcs 值做区间界定：统计三个子数据集的 mcs 差值分布，取最小差值作为统一步长 l_k ，将每个 mcs_k 划入 $[mcs_k - l_k/2, mcs_k + l_k/2]$ 区间，既保留其离散性，又赋予连续边界（如 $mcs = 5.3$ 、 $l_k = 0.5$ 时，区间为 $[5.05, 5.55]$ ）。接着对区间做归一化：统计 A 数据集中 mcs 的最值 mcs_{min} 、 mcs_{max} ，通过公式 (11)，将原始区间映射到 $[0,1]$ ，使其与 EESM 模型输出的归一化速率 $rate \in [0,1]$ 对应，为目标函数提供统一尺度。

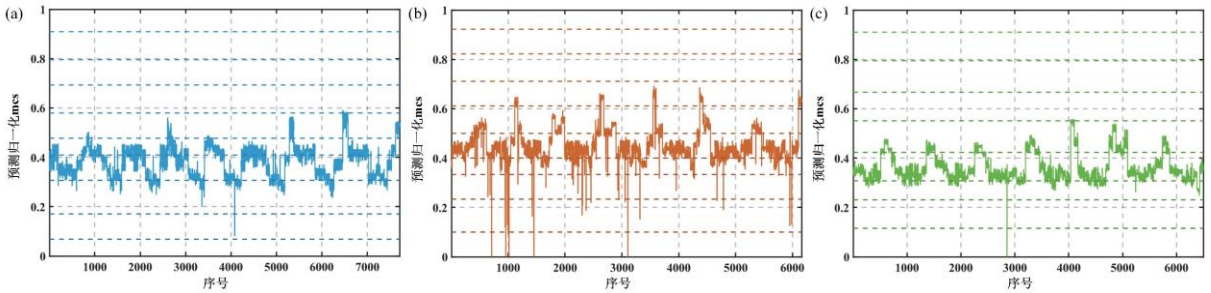


图 18 (a)-(c)三个子数据集预测得到的归一化 mcs

对于传输速率的预测，首先，对数据集中 A 的每条样本按照实际传输速率估计方法计算各子载波的 SINR 值，接着，基于 EESM 模型的映射关系处理这些 SINR 值，建立结合惩罚机制的双边误差平方和形式的目标函数，利用 Adam 优化算法求解目标函数的最小值，最终得到每个子数据集的等效映射系数 α 与 SINR 缩放系数 β 。

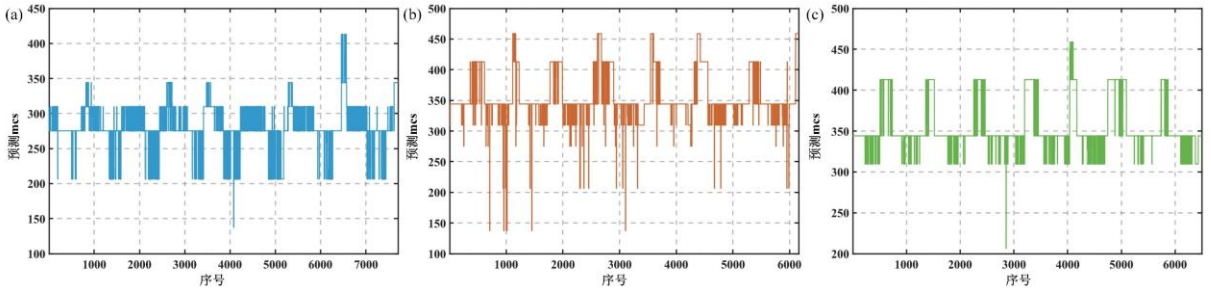


图 19 (a)-(c)三个子数据集预测得到的 mcs

利用该模型对数据集 B 进行计算与预测，得到归一化速率的预测结果，如图 18，之后按照速率索引划分原则将归一化速率映射回原始 mcs 空间，完成传输速率索引预测，如图

19。

3.5 问题小结

本章围绕关闭 TxBF 条件下 MIMO-OFDM 系统的链路速率建模问题，从数据预处理、模型构建与优化到速率预测展开了系统研究。首先通过 IQR 方法剔除信道状态信息与传输速率采集时间差异异常的样本，提高了数据时间同步性；随后建立带约束的目标函数，考虑 mcs 值的区间特性，采用 Adam 自适应优化算法求解 EESM 模型参数 α 和 β ，并针对三个终端分别训练模型以适配各自硬件特性与信道环境。实验结果显示，三个终端模型在 150 轮迭代后均稳定收敛，验证集预测准确率分别达到 70.97%、66.94% 和 61.94%。在 SINR 计算方面，各子数据集的 SINR 值主要集中在 40 左右，且信道间质量差异较小。最终通过计算各子载波 SINR、基于 EESM 模型得到等效 SINR、将归一化速率映射回原始 mcs 空间的完整流程，实现了对未知数据的传输速率索引预测，为自适应传输与资源优化提供了可靠的技术支撑。

4. 问题二：关闭 TxBF 场景下的新型速率估计模型构建与速率估计

4.1 问题分析

问题二要求在关闭 TxBF 的场景下，基于完备数据集 A 构建一种新的速率估计模型，并利用该模型对不完备数据集 B 进行传输速率预测。与问题一的 EESM 模型相比，这里需要突破传统物理层抽象方法的局限，充分挖掘 MIMO 信道矩阵的内在特征与统计规律。

针对问题二，本文求解思路如图 2。

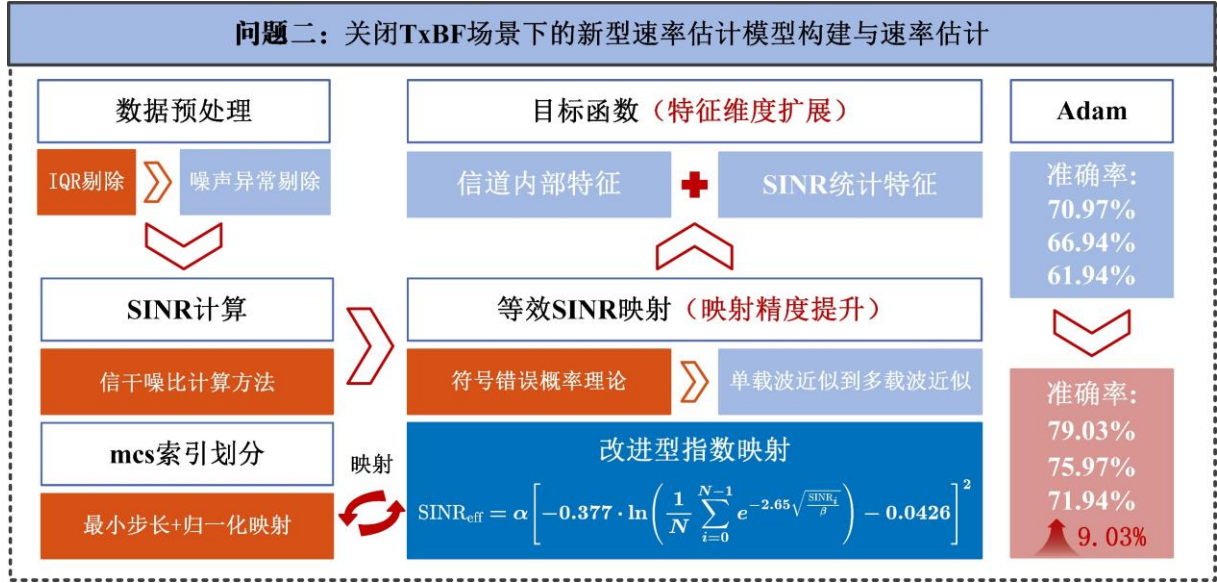


图 20 问题二求解思路

首先，在问题一已完成信道状态信息与传输速率数据时间同步的基础上，针对原始数据中仍存在的噪声异常问题，需进一步开展数据预处理以提升数据纯度，通过 IQR 方法对三个子数据集的噪声水平进行独立分析，依据各终端噪声分布特性划定专属异常阈值，剔除偏离主体分布的尖锐噪声信号，为后续模型训练提供更可靠的输入数据；同时，考虑到传统 EESM 模型中指数映射函数在宽 SINR 范围内近似误差较大的问题，从符号错误概率理论出发，结合多子载波 OFDM 系统总错误概率特性，重构改进型指数映射函数，使其在更宽的 SINR 区间内保持较小近似误差，提升等效 SINR 对整体信道质量的表征精度；另外，针对传统 EESM 模型仅依赖单一 SINR 特征、忽略 MIMO 信道矩阵内在结构特性与 SINR 统计规律的核心缺陷，新型模型需重点突破物理层特征挖掘的局限：一方面对关闭 TxBF 场景下的 $2 \times 4 \times 122$ 维度 MIMO 信道矩阵进行奇异值分解，提取每个载波单元奇异值的均值与方差，以此表征信道传输的能量聚集程度与稳定性；另一方面计算所有载波单元 SINR 的均值、方差及偏度，捕捉子载波间质量波动幅度与分布不对称性，通过特征维度扩展实现对信道质量的多维度刻画，弥补单纯依赖 SINR 无法反映信道细微差异的不足；最后，在模型优化框架上，延续 Adam 自适应优化算法的高效性，但针对新增特征构建含正则化项的目标函数，既保留问题一对非整数 mcs 的区间映射与归一化策略，又通过正则化项平衡特征权重、避免过拟合，确保模型能适配不同终端的硬件特性与信道环境差异，在提升拟合精度的同时保障泛化能力，为后续基于完备数据集 A 的模型验证及不完备数据集 B 的速率估计提供科学、严谨的技术路径。

4.2 数据预处理

4.2.1 噪声异常剔除

经过问题一的求解与模型训练，本文发现在终端收集到的数据中，存在一批噪声异常的数据，故问题二将进行数据再处理。三个子数据集噪声分布如图 21。

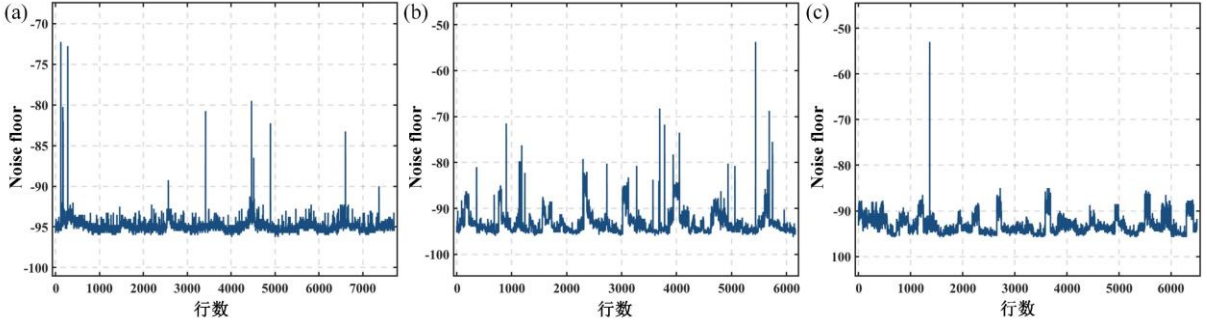


图 21 (a)-(c)三个子数据集的噪声水平分布

从图 21 (a)-(c)可以看出，三个子数据集的噪声水平主要集中在-90 dBm 到-100 dBm 之间，整体分布较为集中且呈近似正态分布特征。然而，数据中也存在少量明显偏离主体分布的尖锐信号，这些信号幅值远高于或低于正常噪声水平，可能源于瞬时干扰、传感器异常响应或数据采集错误。为避

免这些异常值对后续分析与模型训练造成不利影响，需要对其进行剔除处理。与 3.2.1 节同样的流程，我们采用 IQR 对异常数据进行处理，最终得到的数据如表 4。

表 4 数据噪声异常剔除前后大小

终端 MAC 值	原始数据大小（行）	IQR 阈值（dbm）	预处理后大小（行）
341c-f0d4-70be	7475	-96.0000, -94.0000	7018
bc98-2983-907b	6022	-97.7500, -89.7500	5493
d0c1-bfe4-18f0	6197	-97.8750, -88.8750	5896

4.3 模型构建与求解

4.3.1 基于改进型指数映射的等效 SINR 模型

在关闭 TxBF 场景下，针对传统 EESM 模型仅依赖 SINR 而忽略信道矩阵奇异值分布与统计特征的局限，本文提出一种基于改进型指数映射的等效 SINR 模型。该模型从符号错误概率理论出发，在高 SINR 条件下单载波符号错误概率可近似为：

$$P_e(\text{SINR}) \approx e^{-\text{SINR}} \quad (21)$$

对于多子载波 OFDM 系统，总错误概率近似为各子载波错误概率之和：

$$P_s \approx \sum_{i=0}^{N-1} e^{-\text{SINR}_i} \quad (22)$$

为建立等效 SINR 映射关系，引入改进的指数映射函数，该函数相比传统模型能在更宽的 SINR 范围内保持较小近似误差：

$$\phi(\text{SINR}) = e^{-2.65\sqrt{\text{SINR}}} \quad (23)$$

令以下式子成立：

$$\sum_{i=0}^{N-1} \phi(\text{SINR}_i) = N \cdot \phi(\text{SINR}_{eff}) \quad (24)$$

可解出：

$$SINR_{eff} = \phi^{-1} \left(\frac{1}{N} \sum_{i=0}^{N-1} \phi(SINR_i) \right) \quad (25)$$

考虑到实际系统中调制方式、编码率等因素影响，进一步引入映射系数 α 和缩放系数 β 进行修正，得到最终表达式：

$$SINR_{eff} = \alpha \left[-0.377 \cdot \ln \left(\frac{1}{N} \sum_{i=0}^{N-1} e^{-2.65 \sqrt{SINR_i / \beta}} \right) - 0.0426 \right]^2 \quad (26)$$

该模型具有适用性广、精度高等特点，映射系数 α 和缩放系数 β 分别反映等效 SINR 的整体缩放和子载波 SINR 的归一化程度。相比问题一中的传统 EESM 模型，该改进模型能够更充分利用信道信息，显著提升速率估计的准确性。

4.3.2 目标函数

在问题一基于 SINR 与 mcs 区间构建的双边误差平方和目标函数基础上，针对改进型指数映射等效 SINR 模型需融合 MIMO 信道矩阵奇异值分布、SINR 统计特征的需求，首先引入信道矩阵奇异值相关特征与 SINR 统计特征，对关闭 TxBF 场景下的 $2 \times 4 \times 122$ 维度 MIMO 信道矩阵，通过奇异值分解得到每个载波单元对应的奇异值向量，因接收天线数为 2，故奇异值数量为 2：

$$\sigma_i = [\sigma_{i1}, \sigma_{i2}] \quad (27)$$

计算奇异值的均值与方差以表征信道矩阵的能量集中程度与波动特性：

$$\bar{\sigma}_i = \frac{1}{2} \sum_{j=1}^2 \sigma_{ij} \quad (28)$$

$$\sigma_{\sigma_i}^2 = \frac{1}{2} \sum_{j=1}^2 (\sigma_{ij} - \bar{\sigma}_i)^2 \quad (29)$$

同时计算每个样本 122 个载波单元 SINR 的均值、方差及偏度，以捕捉 SINR 的整体水平、离散程度与分布不对称性。

$$\overline{SINR} = \frac{1}{122} \sum_{i=1}^{122} SINR_i \quad (30)$$

$$\sigma_{SINR}^2 = \frac{1}{122} \sum_{i=1}^{122} (SINR_i - \overline{SINR})^2 \quad (31)$$

$$S_{SINR}^2 = \frac{1}{122 \sigma_{SINR}^3} \sum_{i=1}^{122} (SINR_i - \overline{SINR})^3 \quad (32)$$

结合上述新增特征与 mcs 区间的关联关系，构建新的目标函数：首先通过线性组合将奇异值均值、奇异值方差、SINR 均值、SINR 方差及 SINR 偏度转化为特征综合项：

$$F = w_1 \bar{\sigma}_i + w_2 \sigma_{\sigma_i}^2 + w_3 \overline{SINR} + w_4 \sigma_{SINR}^2 + w_5 S_{SINR}^2 \quad (33)$$

将其加入目标函数：

$$J_{min} = \sum_{k=1}^{rows} (\phi(SINR_{eff}^k) - upper^k)^2 + \sum_{k=1}^{rows} (\phi(SINR_{eff}^k) - lower^k)^2 + \lambda \cdot P + \gamma \cdot F \quad (34)$$

其中 γ 为正则化系数， γ 在第三章模型建立与求解的基础上，通过网格搜索确定最优取值，以精准平衡目标函数拟合精度与过拟合风险，提升模型核心性能。

传统 EESM 模型仅依赖 SINR 信息，忽略了 MIMO 信道矩阵的内在结构特性与统计分布特征，而这些特征恰恰能够更精细地表征信道质量与速率之间的非线性关系，通过这些高维度特征线性组合并融入目标函数，可以显著提升模型对不同信道条件的表征能力与鲁棒性，同时通过正则化有效避免过拟合，确保在三个子数据集上均能稳定收敛并保持较高预测精度。这一改进既延续了问题一对非整数 mcs 的区间映射与归一化思路，又通过引入更丰富的物理层特征弥补了单纯依赖 SINR 的不足，该目标函数能够更精准地建立信道特性与传输速率的映射关系，为后续的速率估计提供更可靠的基础。

4.4 结果分析

4.4.1 模型训练

问题二延续按子数据集独立训练的策略，核心原因在于数据预处理与模型特征的特殊性，经噪声异常剔除后，三个终端的有效数据量分别为 7018 行、5493 行、5896 行，数据纯度显著提升，且各终端的噪声基底、信道波动特性差异更突出，同时问题二引入的 MIMO 信道矩阵奇异值特征（均值、方差）与 SINR 统计特征（均值、方差、偏度）在不同终端中呈现强异质性，合并训练会“拉平”终端专属特征与新增特征的差异，导致模型适配性下降。训练参数与超参数设置上，问题二基于问题一的 Adam 自适应优化算法框架，针对改进模型特性调整超参数，核心优化目标为最小化带约束的目标函数式(34)。

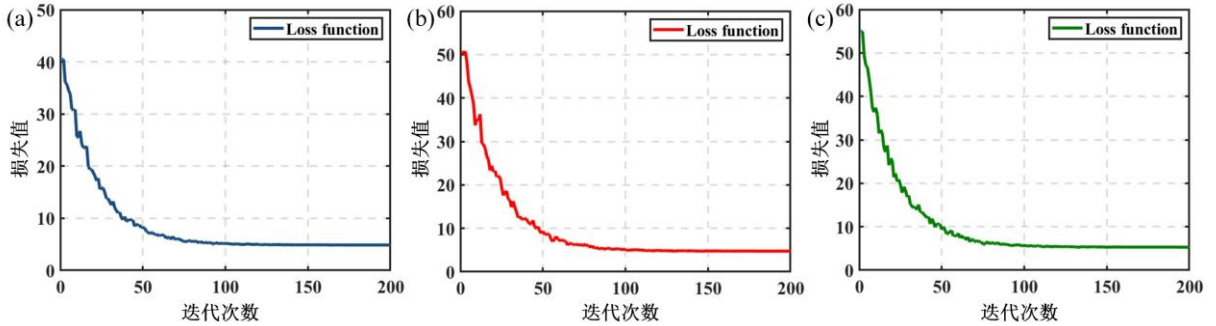


图 22 (a)-(c)训练过程中三个子数据集的损失函数变化

训练过程如图 22，迭代次数设为 250 轮，基础学习率沿用 $\eta = 0.0001$ ，惩罚系数 λ 保持 0.1，正则化系数 γ 通过 5 折交叉验证确定为 0.05，特征权重 $w_1 - w_5$ 初始化为 0.2，Adam 动量参数 $\beta_1 = 0.9$ 、 $\beta_2 = 0.999$ ，数值稳定性常数 $\epsilon = 10^{-8}$ 。训练过程监控中，三个子数据集的损失函数值在训练初期（1~50 轮）快速下降，50~150 轮进入参数微调阶段，150 轮后趋于稳定，200 轮迭代时完全收敛。

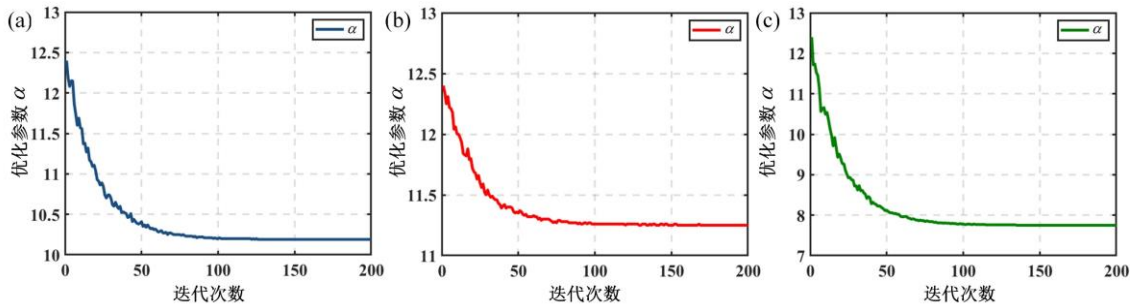


图 23 (a)-(c)训练过程中优化参数 α 的变化

待优化参数的训练过程如图 23 与 24 中，基础参数 α ，最终收敛值为 10.19, 11.25, 7.75, β 收敛值为 783.59、755.19、796.06，特征权重 w_1 （奇异值均值）最大（0.42~0.48）， w_3 （SINR 均值）次之（0.25~0.30）， w_2 、 w_4 、 w_5 权重较小（0.05~0.15）。200 轮迭代后，三个子数据集的最终训练结果中，正则化系数 γ 统一设为 0.05。

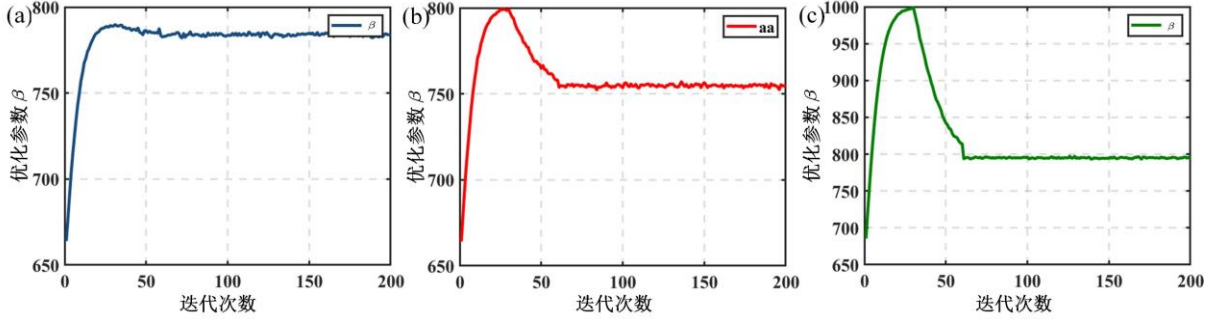


图 24 (a)-(c)训练过程中优化参数 β 的变化

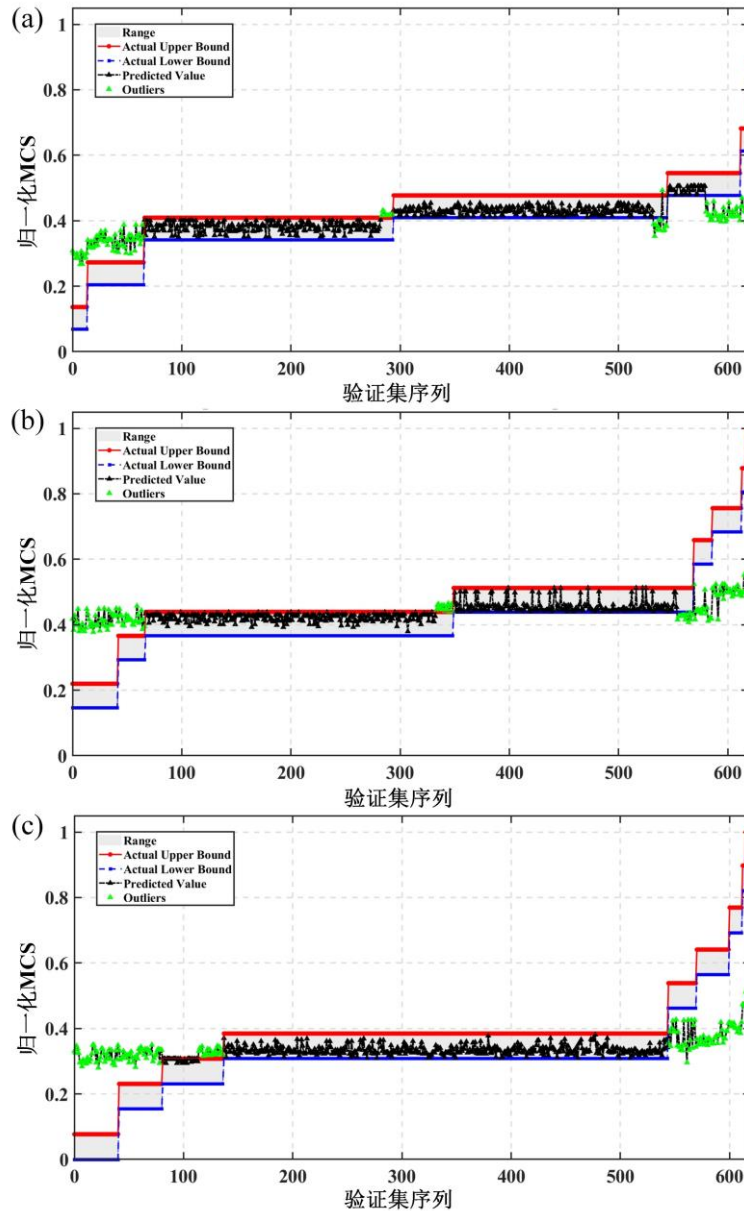


图 25 (a)-(c)三个子数据集的验证集预测效果

验证集性能如图 25，采用与问题一相同的 9:1 划分策略（验证集约 620 个样本），问题二模型的验证集预测准确率分别为 **79.03%、75.97%、71.94%**，较问题一平均提升 **9.03%**，核心原因包括特征维度扩展、映射函数改进与数据质量提升。

4.4.2 基于改进型指数映射模型的等效 SINR 映射

基于改进型指数映射模型的等效 SINR 映射是将多载波系统中各子载波上的 SINR 值转化为一个能够代表整体信道质量的单一指标，本文采用改进型指数映射模型模型进行拟合计算，将 4.4.1 节中拟合得到的优化参数 α 与 β 代入 4.3.1 节的公式(26)，得到三个子数据集的等效 SINR 如图 26。

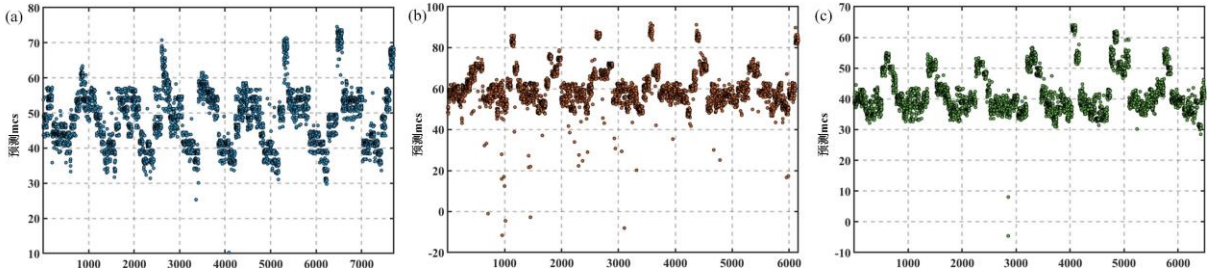


图 26 (a)-(c)三个子数据集预测得到的归一化 mcs

4.3.3 基于改进型指数映射模型的传输速率预测

类似于 3.4.4 节中传输速率预测，本文针对问题二提出了一种新的基于改进型指数映射与多特征约束的传输速率预测模型，该模型相较于问题一模型，核心优势体现在映射精度提升与特征维度扩展，本模型新增 MIMO 信道奇异值特征（均值、方差，表征信道能量集中度与稳定性）和 SINR 统计特征（均值、方差、偏度，捕捉子载波间质量差异与分布特性），本模型改进指数映射函数，结合推导的反函数计算等效信干噪比，在更宽的范围内保持小近似差，使信道整体质量表征更精准；同时通过特征综合项融入目标函数，更精细地表征信道质量与速率的非线性关联，避免问题一特征维度不足导致信道表征片面。

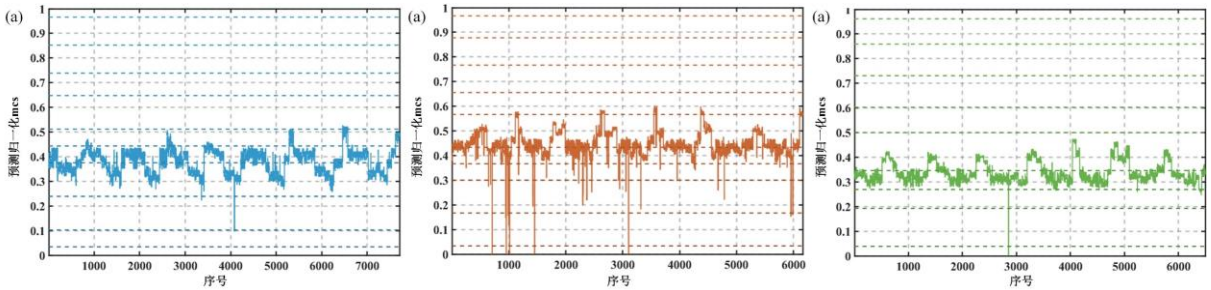


图 27 (a)-(c)三个子数据集预测得到的归一化 mcs

利用该模型对数据集 B 进行计算与预测，得到归一化速率的预测结果，如图 27，之后按照速率索引划分原则将归一化速率映射回原始 mcs 空间，完成传输速率索引预测，如图 28。

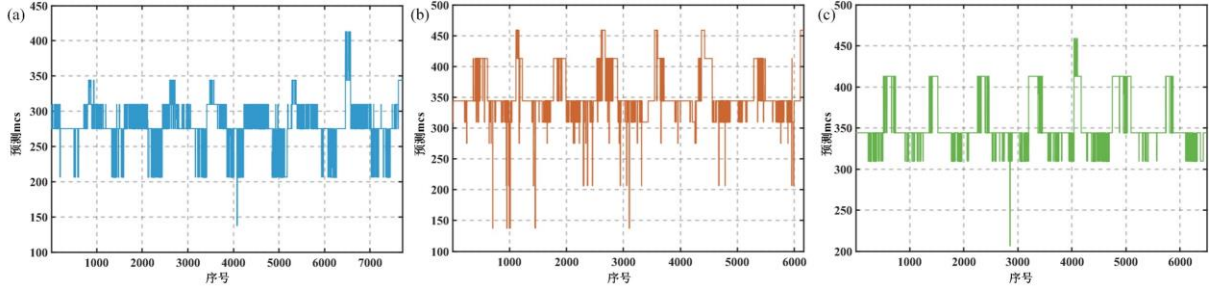


图 28 (a)-(c)三个子数据集预测得到的mcs

4.5 问题小结

本节聚焦关闭 TxBF 场景下 MIMO-OFDM 系统的新型速率估计问题,在问题一信道与速率数据时间同步处理的基础上,进一步优化数据质量并突破传统模型局限,完成映射精度提升与特征维度扩展的系统性研究。

首先在数据预处理阶段,针对原始数据中存在的瞬时干扰、传感器异常等噪声异常问题,采用 IQR 方法对三个子数据集的噪声水平进行针对性异常剔除。依据各终端噪声分布特性划定专属 IQR 阈值,为后续高精度模型训练提供更纯净的数据支撑。

在映射精度提升方面,针对传统 EESM 模型中指数映射函数在宽 SINR 范围内近似误差较大、信道质量表征精度不足的局限,本文从符号错误概率理论出发,重构映射函数以提升等效 SINR 的计算精度。

在特征维度扩展方面,传统 EESM 模型仅依赖单一的 SINR 特征,忽略了 MIMO 信道矩阵的内在结构特性与 SINR 的统计分布规律,导致对复杂信道条件的表征能力不足。本文通过扩展特征维度,引入两类关键物理层特征,并构建特征综合项融入模型,实现对信道质量的多维度刻画。

实验结果显示,三个子数据集模型均在 200 轮迭代后稳定收敛,特征权重分析表明,奇异值均值权重最大, SINR 均值次之,验证了新增物理层特征对速率估计的关键作用。在 9:1 划分的验证集上,模型预测准确率分别达 **79.03%**、**75.97%**、**71.94%**,较问题一传统 EESM 模型平均提升 **9.03%**。该新型模型有效解决了传统方法对信道内在结构特征利用不足的问题,显著提升关闭 TxBF 场景下速率估计的鲁棒性与精度,为无线通信系统的自适应传输、资源调度及链路风险管控提供更可靠的技术支撑。

5. 问题三：开启 TxBF 场景下的新型速率估计模型构建与速率估计

5.1 问题分析

问题三的核心任务是针对开启 TxBF 的 MIMO-OFDM 系统，基于完备数据集 A 中的信道矩阵与对应传输速率标签，构建适用于该场景的新型速率估计模型，并利用该模型对不完备数据集 B 的同场景信道矩阵进行传输速率估计。其本质是突破“关闭 TxBF 场景”的信道特征局限，适配 TxBF 技术带来的信道结构变化与传输机制差异，解决传统模型在波束赋形开启时估计精度大幅下降的问题。

针对问题三，本文的求解思路如图 29。

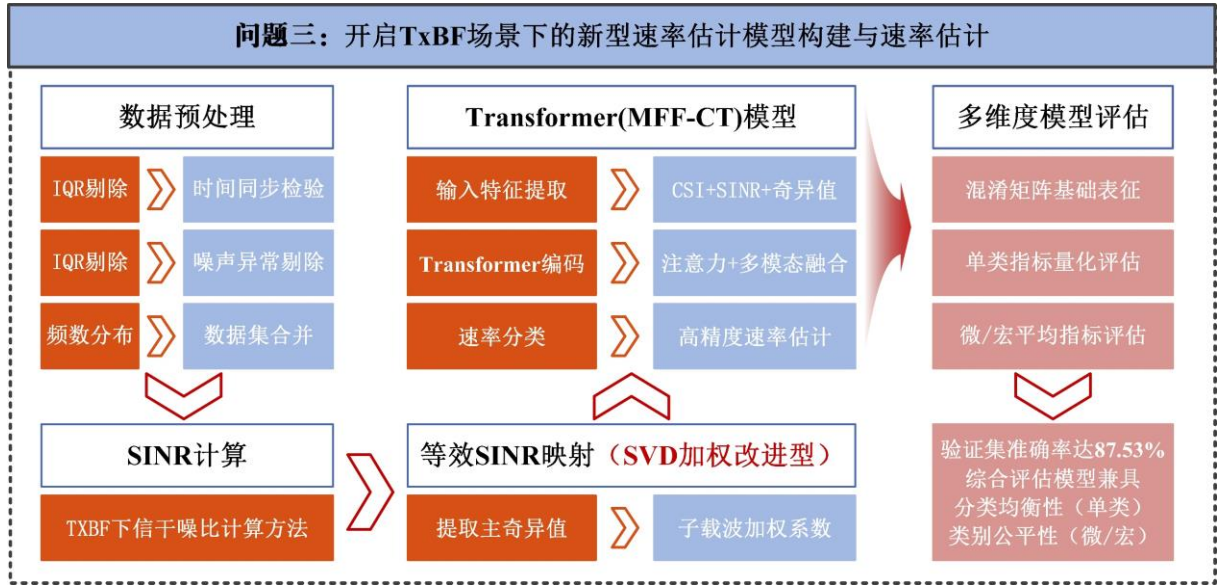


图 29 问题三求解思路

5.2 数据预处理

对于题目所给的开启 TxBF 的六个数据集(341c-f0d4-70be, 508e-4963-d5dc, 508e-4963-e122, bc98-2983-907b, d0c1-bfe4-18f0, fc02-9668-e2fd)，我们首先进行同步时间差特征分布分析：

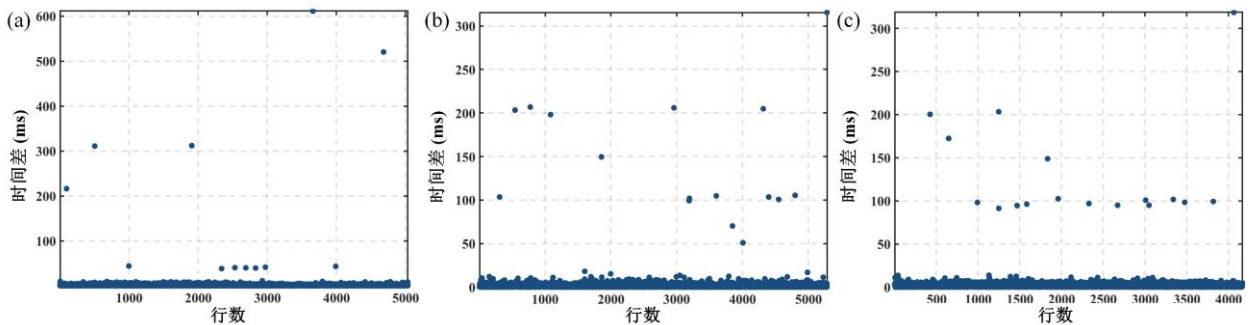


图 30 (a)-(c)前三个数据集的时间差分布图

从图 30 中可以发现，开启 TxBF 收集的数据中仍然存在收集时间不对准的情况，意味着该数据集也需要进行数据采集时间同步检验。

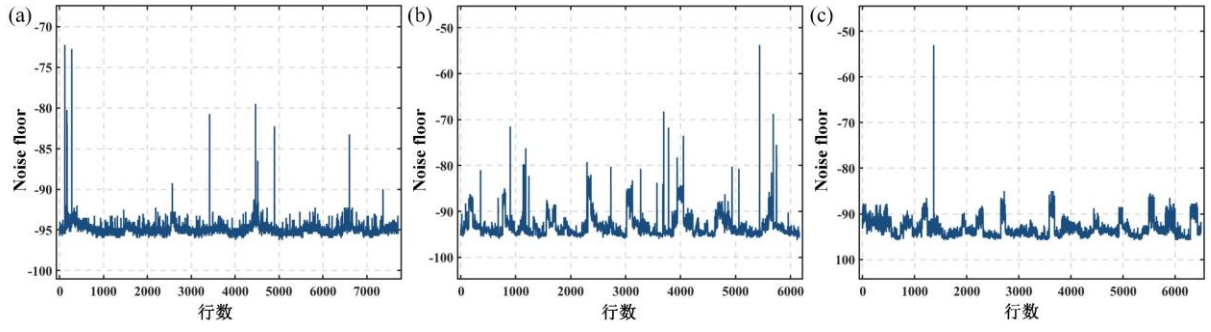


图 31 (a)-(c)三个子数据集的噪声水平分布

按照 4.2.1 节同样的思路，对于数据集中的噪声水平分布进行分析如图 31，从图中可以发现数据中的噪声水平也存在异常的情况，意味着该数据集也需要进行噪声异常剔除。

同时，考虑到问题三中子数据集比较多，同时考虑到训练效果，故将六个数据集进行整体合并，然后再进行数据预处理的操作，最终得到数据大小如表 5。

表 5 合并数据预处理前后大小

终端 MAC 值	合并之后大小	IQR 阈值 1 (ms)	IQR 阈值 2 (dbm)	预处理后大小
341c-f0d4-70be	27549	[-2.8320, 5.7280]	[-100.125, -91.125]	25638
508e-4963-d5dc				
508e-4963-e122				
bc98-2983-907b				
d0c1-bfe4-18f0				
fc02-9668-e2fd				

5.3 模型构建与求解

5.3.1 基于 SVD 加权改进型指数映射的等效 SINR 模型

对于前两问未开启 TxBF 的情况，此时 MIMO-OFDM 系统仅利用多天线的空间复用能力提升速率，无需对发送信号进行相位旋转与相干增强处理，信号传输仅依赖信道矩阵本身的频率域特征即各载波单元的信道系数分布。而针对第三问开启了 TxBF 的情况，TxBF 的核心作用是通过每根发送天线的信号进行相位旋转，利用电磁波的相干原理增强特定方向的信号强度，本质是在 MIMO-OFDM 基础上叠加了空间域的信号优化，需额外考虑波束赋形相关的信号处理逻辑。

SINR 是速率估计的核心输入，MIMO-OFDM 系统的 SINR 仅依赖信道矩阵的范数，即式(1)，开启 TxBF 后，SINR 计算需引入波束赋形权值矩阵 V_i ，即来自信道矩阵 SVD（奇异值）分解的西矩阵，例如第 i 个载波单元上 $N \times M$ 维的信道矩阵 H_i 进行 SVD 分解：

$$H_i = U_i D_i V_i^H \quad (35)$$

其中 U_i 是 $N \times N$ 维西矩阵， D_i 为一个 $N \times M$ 维对角矩阵， V_i 是 $M \times M$ 维西矩阵。TxBF 的 MIMO 系统在第 i 个载波单元上的接收信号的数学表达为：

$$y_i = H_i V_i s_i + n_i \quad (36)$$

其中 $y_i = [y_1, \dots, y_N]^T$ ， $V_i = [w_1, \dots, w_M]^T$ ， $s_i = [s_1, \dots, s_M]^T$ ，其中 w_k 为旋转并缩放列向量，也即权值向量。使得 w_k 旋转 s_k 对应到主奇异值上以获得最大的传输速率。

由于信道矩阵 H_i 会具有多个奇异值，因此信道矩阵可以传输多个独立的发送信号，即：

$$y_i = \sum_{l=0}^{L-1} H_i w_{i,k} s_{i,k} + n_i \quad (37)$$

其中 L 由矩阵的秩决定，表示 MIMO 信道可传输发送信号的数量，而 $\mathbf{w}$ 是彼此正交的列向量，其中 $k \in \{0, L-1\}$ 。

由于开启波束赋形后，信号会在最大的奇异值对应的方向上得到最大增益，将奇异值的最大值称为主奇异值，即：

$$\sigma_{i,max} = \max(\sigma_i) \quad (38)$$

主奇异值不同，信号增益不同，故不能沿用前文对应相同的缩放系数。对于开启 TxBF 后同一个样本，计算其 SINR 值如图 32，可以发现开启 TxBF 后其 122 个子载波的 SINR 值发生了显著变化，证明开启 TxBF 后各子载波增益效果不同，即其奇异值存在显著差异。

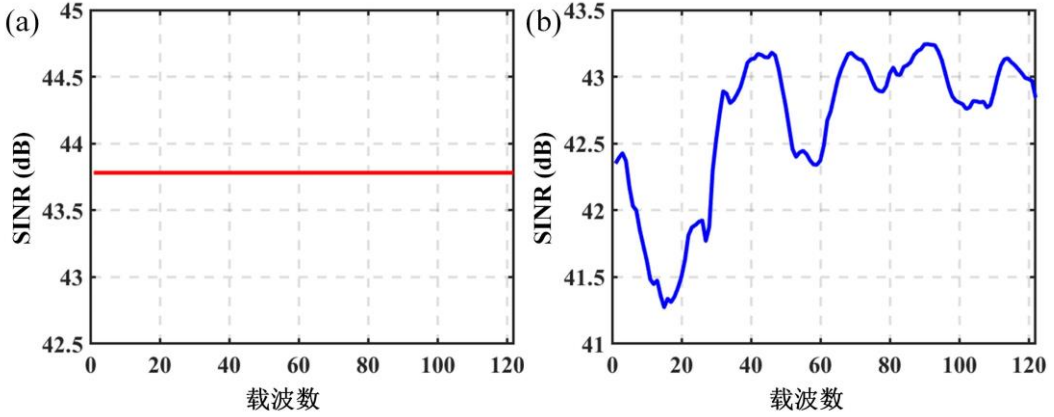


图 32 (a),(b) SVD 分解前后计算得到的 SINR

基于上述分析，本文在问题二改进型指数映射模型的基础上，引入基于主奇异值占比的加权系数，构建适用于开启 TxBF 场景的等效 SINR 模型。其核心思路是通过各子载波主奇异值在所有子载波主奇异值总和中的占比，来定义该子载波的速率贡献权重，确保权重分配与 TxBF 的传输机制高度契合。具体而言，首先针对每个子载波 i ，从其 SVD 分解得到的奇异值对角矩阵中提取主奇异值，随后计算所有 N 个子载波主奇异值的总和，其代表了整个系统在所有子载波上的总信号增益潜力，最后第 i 个子载波的加权系数可以通过以下公式进行计算：

$$p_i = \frac{\sigma_{i,max}}{\sum_{j=0}^{N-1} \sigma_{j,max}} \quad (39)$$

同时其加权系数应满足以下要求：

$$\sum_{i=0}^{N-1} p_i = 1 \quad (40)$$

即确保加权求和后的结果仍处于合理的数值范围，避免因权重叠加导致的数值溢出或失真。同时，主奇异值越大，说明该子载波的信号传输增益越高、抗干扰能力越强，对整体速率的贡献自然越大，通过 p_i 的权重分配，能够让等效 SINR 计算过程充分体现这种贡献差异，有效规避因权重均等化导致的模型偏差。

在确定加权系数 p_i 的基础上，结合问题二中基于符号错误概率理论推导的改进型指数映射函数，可进一步推导开启 TxBF 场景下的等效 SINR 计算公式如下：

$$SINR_{eff} = \left[-0.377 \cdot \ln \left(\frac{1}{N} \sum_{i=0}^{N-1} e^{-2.65 \sqrt{SINR_i / \beta}} \right) - 0.0426 \right]^2 \quad (41)$$

该模型利用了 SVD 分解对 TxBF 信道空间特征的提取能力，还保留了改进型指数映射

在宽 SINR 范围内的高近似精度，为后续开启 TxBF 场景下的速率估计提供了精准、可靠的信道质量量化指标。

5.3.2 基于 CSI 与 SINR 多模态特征融合的信道 Transformer (MFF-CT)模型

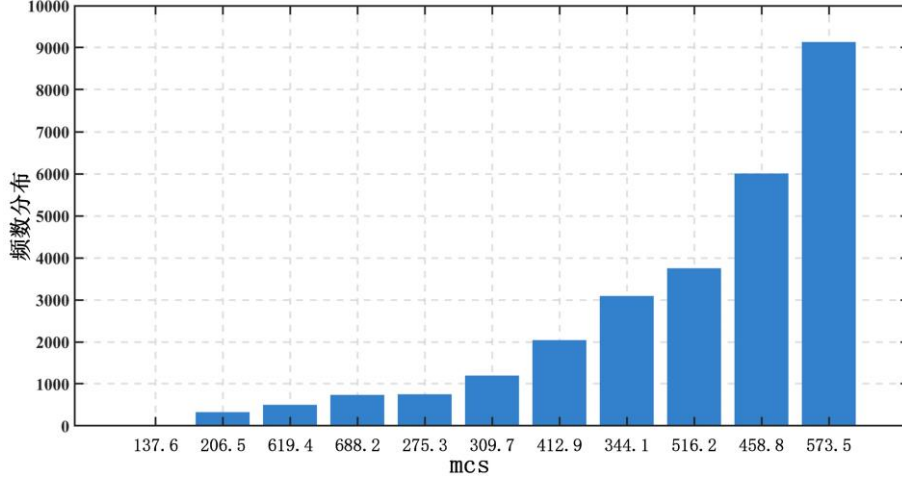


图 33 合并数据集中传输速率mcs频数分布

针对合并数据集 A 中的mcs频数分布可知，其涵盖 11 个 mcs 类别且样本总量庞大，且类别间样本失衡显著，传统的 EESM 模型仅依赖单一 SINR 特征，难以应对样本失衡带来的泛化性挑战，更无法适配 TxBF 场景的空间增益特性，因此亟需构建适配该场景的预测模型以实现高精度速率估计。

问题三聚焦开启 TxBF 的 MIMO-OFDM 场景，而此前问题一的 EESM 模型、问题二的改进型指数映射模型存在显著局限：前者仅依赖单一 SINR 特征，忽略信道空间结构；后者虽扩展了 CSI 奇异值特征，但未适配 TxBF 通过波束赋形权值矩阵实现空间增益，传统模型未融入主奇异值、载波贡献权重等 TxBF 专属特征，且未协同 CSI 的频率、空间细节与 SINR 的全局质量信息，导致速率估计偏差显著。因此，需构建适配 TxBF 场景的新型模型，填补传统方法的场景适配空白。

针对开启 TxBF 场景下 MIMO 信道的空间域波束增益特性与多载波频率域差异，我们在原始 CSI-SINR 融合框架基础上建立基于 CSI 与 SINR 多模态特征融合的信道 Transformer (MFF-CT)模型，通过 TxBF 特征嵌入、加权注意力机制、跨模态全局融合三大核心改进，实现对波束赋形信道特性的建模。模型整体架构分为 5 层：输入特征预处理层、多模态特征嵌入层、加权 Transformer 编码器层、跨模态融合层、速率分类层，具体结构与实现细节如下：

(1) 输入特征预处理层

开启 TxBF 后，信道矩阵的空间增益由波束赋形权值矩阵决定，因此需在原始 CSI 与 SINR 预处理中，融入 SVD 空间特征与加权贡献系数，构建适配场景的多维度输入特征。

输入原始 CSI 数据为三维复数张量 $H \in \mathbb{C}^{N_{rx} \times N_{tx} \times N_{sc}}$ ，其中 $N_{rx} = 2, N_{tx} = 4, N_{sc} = 122$ ，对于每个子载波，首先按照式(35)执行 SVD 分解以提取 TxBF 核心特征，基于 SVD 结果，提取两类 TxBF 专属空间特征，为方便训练，将其与原始 CSI 特征融合，具体步骤如下：

首先延续基础框架的天线维度展平与实虚部分解逻辑，将 H 的天线维度展平为 8 维复数向量，再拼接成 16 维基础特征向量即

$$\mathbf{x}_k = [\text{Re}(h_k), \text{Im}(h_k)] \quad (42)$$

式中 $\mathbf{x}_k$ 向量完整地表征了第 k 个子载波上的全部信道信息，是 Transformer 模型的一个 Token。

接着对于针对每个子载波 i ，提取 SVD 分解得到的主奇异值和加权系数空间特征并标准化，将其与原有的 16 维特征拼接，形成 18 维特征向量：

$$x_k = [Re(h_k), Im(h_k), \sigma_i^{max}, p_i] \quad (43)$$

同时本文考虑 SINR 所携带的信道消息，同时为避免传统单一标量 SINR 的信息损失，仿照 4.3.2 节中 SINR 的多维特征，结合等效 SINR 值，构建 5 维 SINR 特征向量，针对标准化后的 4 维 SINR 特征，通过线性投影扩展到 122 维，作为 SINR 模态的输入特征：

$$x_k = [Re(h_k), Im(h_k), \sigma_i^{max}, p_i, SINR_i, SINR_{eff}, \overline{SINR}, \sigma_{SINR}^2, S_{SINR}^2] \quad (44)$$

输入特征预处理层中完成对 CSI 特征与 TxBF 特征、SINR 多维特征的融合，形成 23 维的特征向量输入。

(2) 统一 Transformer 建模空间

该层的核心是将 CSI 序列特征与 SINR 多维特征分别映射到 Transformer 的统一建模空间，确保多模态特征的可融合性与表达能力。

序列中的每一个输入向量 $x_k \in R^{22}$ 有一个对应的词嵌入向量 $e_k \in R^{d_{model}}$

$$e_k = x_k W_e + b_e \quad (45)$$

其中， $W_e \in R^{22 \times d_{model}}$ 是可学习的权重矩阵， $b_e \in R^{d_{model}}$ 是偏置向量。

原始的自注意力机制是“置换不变”的，也就是打乱序列中元素的顺序不会改变计算结果。但是我们子载波的数据具有物理背景，其顺序代表频率的顺序，物理约束其不可逆转。位置编码就是为序列中的每个词嵌入向量，注入其在序列中的绝对位置信息。使用正弦/余弦位置编码。对于序列中位置为 pos (从 0 到 121)、维度为 i 的编码值：

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \quad (46)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \quad (47)$$

每个位置就因此得到一个独特的、由不同频率的正余弦波组合而成的位置向量 $P_k \in R^{d_{model}}$ ，进而将词嵌入和位置编码的逐元素相加的结果送入模型的编码器

$$Z_0 = E + PE \quad (48)$$

(3) 加权 Transformer 编码器层

原始 Transformer 的自注意力机制对所有子载波“同等对待”，但开启 TxBF 后，主奇异值大的子载波对速率贡献更高。因此引入加权自注意力机制(Weighted Multi-Head Self-Attention, WMHA)，增强模型对高增益载波的关注。编码器采用 4 层堆叠，每层包含“加权多头自注意力”与“前馈网络”两个子模块，且均配备残差连接与层归一化用以确保训练稳定性。

首先是多头自注意力(Multi-Head Self-Attention)该模块用来计算序列中每个元素与其他所有元素之间的关联强度，并以此更新自身的表示。在数学上首先将输入 $Z-1$ 线性变换为三个矩阵:查询(Query)Q，键(Key)K，值(Value)V。对于单个注意力头，其输出计算为：

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (49)$$

本文的模型使用了 $h = 4$ 个注意力头，所以我们将输入并行地进行 4 次独立的注意力计算(使用不同的 Q,K,V 变换矩阵)，然后将 4 个头的输出拼接并再次进行线性变换。这使得模型能同时关注来自不同“表示子空间”的信息。

另一个模块是前馈神经网络(Feed-Forward Network)，使用该模块对自注意力模块的输

出进行一次非线性变换，增加模型的表达能力。数学上这部分由两个全连接层和 ReLU 激活函数组成的简单多层感知机(MLP)，它被独立地应用于序列中的每一个元素。

$$FFN(z) = \text{ReLU}(zW_1 + b_1)W_2 + b_2 \quad (50)$$

为了保证训练的稳定性和效率，每个子模块的输出都经过了残差连接(Residual Connection)和层归一化(Layer Normalization)。

(4) 跨模态特征融合层

采用平均池化(Mean Pooling)的方式对经过 4 层编码后的输出序列进行分类；序列维度为 122，则对 122 个向量取平均值，得到一个能代表整个信道状态的上下文向量：

$$z_{agg} \in R^{d_{model}} \quad (51)$$

然后融合 SINR_{eff} ，将标准化后的 SINR_{eff} 标量特征 $s' \in R$ 与上下文向量进行拼接，形成最终的融合特征向量：

$$z_{\text{final}} \in R^{d_{model}+1} \quad (52)$$

在该网络中接收融合后的 z_{final} 向量，并将其映射到最终的 11 个类别的“分数”(Logits)，以供交叉熵损失函数计算和最终的分类决策。

(5) 速率分类层

分类层的核心是将 256 维融合特征映射到实际的 mcs 索引类别(一共 11 类)，实现开启 TxBF 场景下的速率估计。训练时，采用交叉熵损失函数：

$$L = \frac{1}{N} \sum_i L_i = -\frac{1}{N} \sum_i \sum_{c=1}^M y_{ic} \log(p_{ic}) \quad (53)$$

最终通过全连接层将 z_{final} 映射到实际的 mcs 索引类别，实现开启 TxBF 场景下的速率估计。

MFF-CT 模型专为开启 TxBF 的 MIMO-OFDM 系统设计，用于速率估计，分五层架构：输入层融合 CSI 原始信息、TxBF 专属的主奇异值和子载波加权系数、SINR 多维特征；再通过词嵌入和位置编码统一建模空间；加权 Transformer 编码器用 WMHA 机制重点关注高增益子载波，契合 TxBF 定向增益特性；跨模态融合层整合序列特征与等效 SINR 全局信息；最后由速率分类层输出 mcs 索引。该模型将 TxBF 物理机制融入特征和注意力设计，解决传统模型不重视空间域的问题，适配该场景的高精度速率估计需求。

5.3.3 模型训练

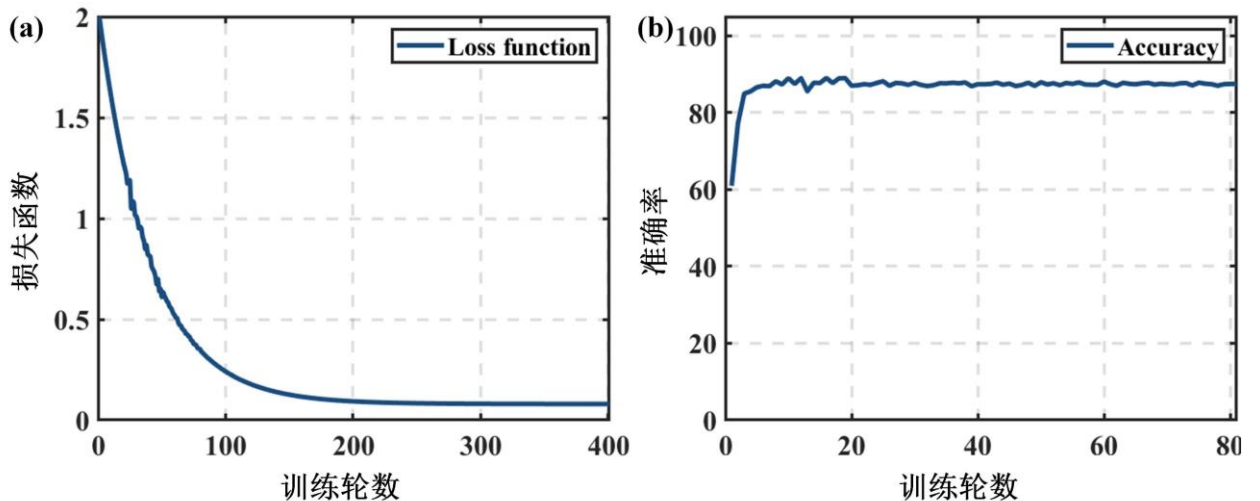


图 34 (a)模型训练的损失函数变化；(b)训练过程中验证集的准确率

针对开启 TxBF 场景的 MFF-CT 模型训练，先将预处理后 25638 条有效样本按 8:2 分层抽样为验证集，采用交叉熵损失，每轮前向传播输出 mcs 类别、反向传播更新参数并进行梯度裁剪；训练中前 50 轮损失与准确率快速提升，50-150 轮微调，150-200 轮收敛，200 轮后验证集准确率达 **87.53%**，较关闭 TxBF 场景的模型准确率也显著提升。

5.4 结果分析

5.4.1 SINR 计算

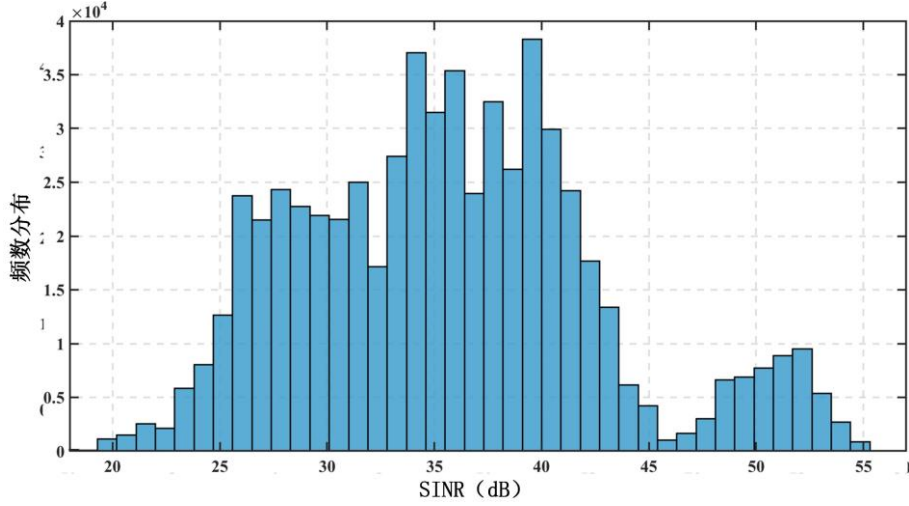


图 32 合并数据集中 SINR 频数分布

针对问题三给的数据集，我们将其进行合并按照开启 TxBF 下计算公式计算：

$$SINR_i = \|H_i V_i\|^2 / \|n_i\|^2 \quad (54)$$

最终得到合并数据集中 SINR 频数分布如图 32。

5.4.2 等效 SINR 计算

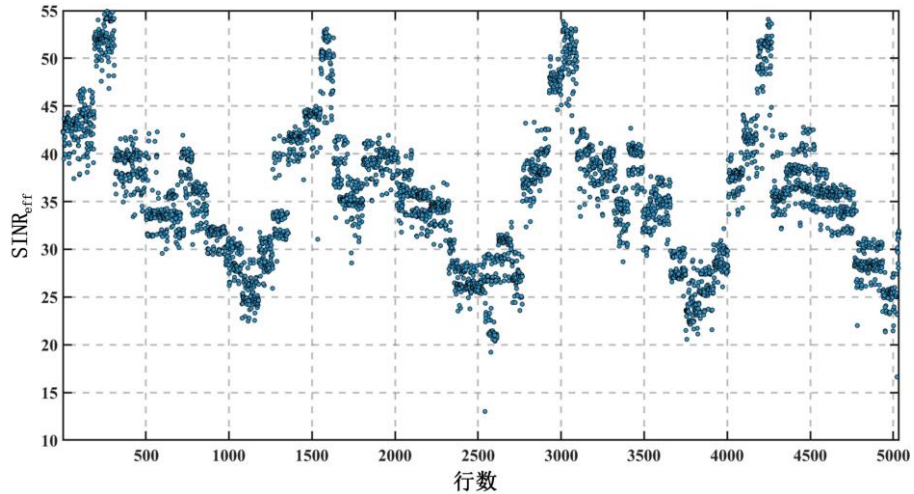


图 33 合并数据集中部分等效 SINR 值

在 5.4.1 节基础上，结合基于 SVD 加权改进型指数映射的等效 SINR 模型式(41)，得到等效 SINR 值结果如图 33。

5.4.3 mcs 预测

基于构建的 CSI 与 SINR 多模态特征 Transformer (MFF-CT) 模型预测结果如图 33。

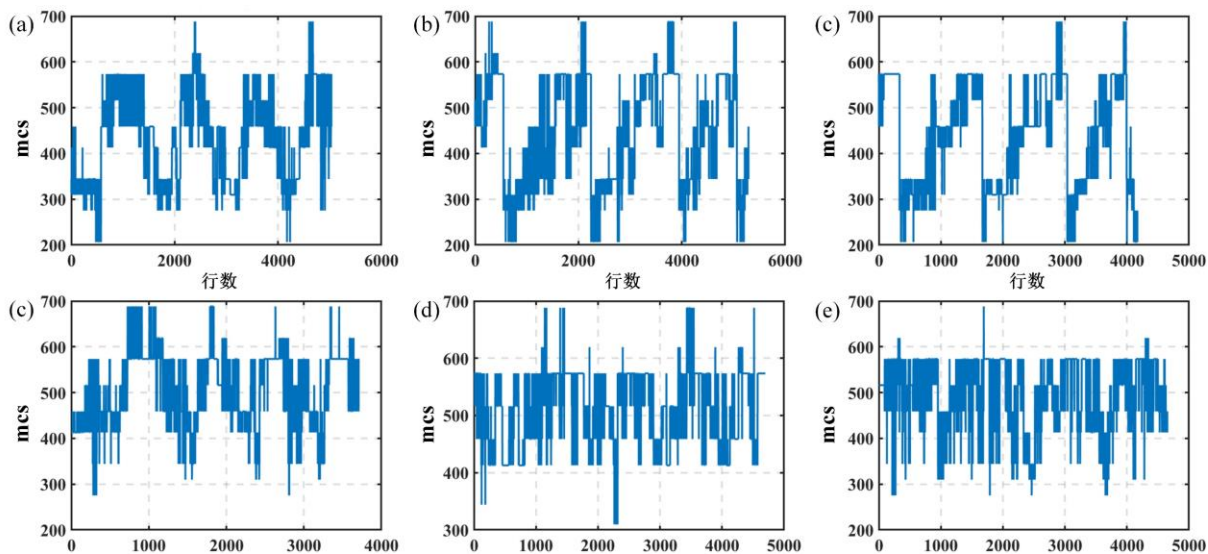


图 33 (a)-(d)六个子集的mcs预测结果

5.4.4 模型评估

为全面量化多分类任务模型的性能，本论文构建基础表征、单类量化、综合评估的三级指标体系，以混淆矩阵为基础工具，提取核心统计量；基于统计量计算精确率、召回率与特异度，实现单类性能量化；通过微平均与宏平均指标，从全局与类别维度完成综合评估，确保结果的客观性与全面性。

(1)混淆矩阵:

混淆矩阵是直观呈现模型预测结果与真实类别的匹配关系的核心工具，为所有衍生指标提供原始数据支撑。在 11 类多分类任务中，混淆矩阵为 11×11 方阵，矩阵的行对应样本真实类别，列对应模型预测类别，元素 (i, j) 代表“真实归属第 i 类且被预测为第 j 类”的样本数量，本模型预测得到的混淆矩阵如图 34。

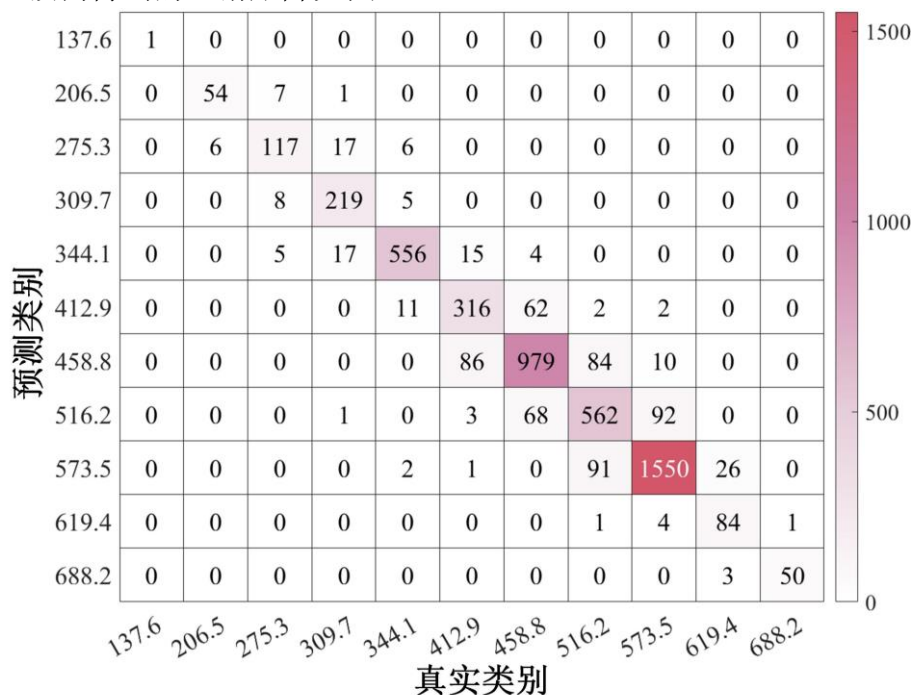


图 34 混淆矩阵

基于混淆矩阵，定义四类核心统计量（以第*i*类为例）：

真阳性 TP_i ：真实为第*i*类且预测为第*i*类的样本数，对应矩阵对角线元素(*i*,*i*)，反映模型对第*i*类的正确识别数量；

假阳性 FP_i ：真实非第*i*类但预测为第*i*类的样本数，对应矩阵第*i*列非对角线元素之和，反映模型将其他类误判为第*i*类的数量；

真阴性 TN_i ：真实非第*i*类且预测非第*i*类的样本数，对应矩阵中除第*i*行与第*i*列外所有元素之和，反映模型对“非第*i*类”的正确排除数量；

假阴性 FN_i ：真实为第*i*类但预测非第*i*类的样本数，对应矩阵第*i*行非对角线元素之和，反映模型将第*i*类漏判为其他类的数量。

通过混淆矩阵可直接观察模型的分类偏差，例如某类 TP_i 占比高且 FP_i 、 FN_i 占比低，表明模型对该类的分类性能优异；若某类 FN_i 显著偏高，则提示模型对该类的识别能力不足。

(2)基础单类指标：

精确率（Precision）：

精确率聚焦“预测结果的纯度”，定义为“被预测为第*i*类的样本中，真实归属第*i*类的样本占比”，用于评估模型预测正类的准确性。其计算公式为：

$$Precision_i = \frac{TP_i}{TP_i + FP_i} \quad (55)$$

式中，分子为第*i*类的正确预测数 TP_i ，分母为所有被预测为第*i*类的样本数 TP_i 与 FP_i 之和。

在多分类任务中，需分别计算每个类别的精确率，再通过算术平均得到全局宏平均精确率。本研究宏平均精确率为 **0.8745**，表明在所有被预测为各类别的样本中，平均约 87.45% 为真实归属该类的样本，模型预测结果的“纯度”较高。

召回率（Recall）：

召回率聚焦“真实正类的覆盖能力”，定义为“真实归属第*i*类的样本中，被正确预测为第*i*类的样本占比”，用于评估模型对真实正类的捕获能力。其计算公式为：

$$Recall_i = \frac{TP_i}{TP_i + FN_i} \quad (56)$$

式中，分子为第*i*类的正确预测数 TP_i ，分母为所有真实归属第*i*类的样本数 TP_i 与 FN_i 之和。

本研究宏平均召回率为 **0.8887**，表明在所有真实归属各类别的样本中，平均约 88.87% 被模型正确识别，模型对真实正类的“覆盖能力”较强。

特异度（Specificity）：

特异度聚焦“负类的识别能力”，定义为“真实非第*i*类的样本中，被正确预测为非第*i*类的样本占比”，用于评估模型对负类的排除能力。其计算公式为：

$$Specificity_i = \frac{TN_i}{TN_i + FP_i} \quad (57)$$

式中，分子为第*i*类的正确排除数 TN_i ，分母为所有真实非第*i*类的样本数 TN_i 与 FP_i 之和。本研究宏平均特异度为 **0.9862**，表明在所有真实非第*i*类的样本中，平均约 98.62% 被模型正确排除，模型对负类的“甄别能力”优异。特异度与召回率形成互补关系：召回率衡量模型对正类的识别能力，特异度衡量模型对负类的识别能力，二者结合可全面反映模型对“正负类边界”的区分精度。

(3)微平均与宏平均指标：

微平均指标：

微平均指标的核心逻辑是“先汇总所有类别的统计量，再计算全局指标”，将每个样本视为同等重要的评估单元，适用于判断模型对全局样本的整体分类均衡性。

计算过程如下：

对 11 类的 TP_i 、 FP_i 、 FN_i 分别求和，得到总真阳性 $Total TP$ 、总假阳性 $Total FP$ 、总假阴性 $Total FN$ ：

$$Total TP = \sum_{i=1}^{11} TP_i \quad (58)$$

$$Total FP = \sum_{i=1}^{11} FP_i \quad (59)$$

$$Total FN = \sum_{i=1}^{11} FN_i \quad (60)$$

微平均精确率代表衡量全局预测正类的纯度，计算公式如下：

$$Micro - Precision = \frac{Total TP}{Total TP + Total FP} \quad (61)$$

代入数据计算得，微平均精确率为 **0.8750**；

微平均召回率代表衡量全局真正正类的覆盖能力，计算公式如下

$$Micro - Recall = \frac{Total TP}{Total TP + Total FN} \quad (62)$$

代入数据计算得，微平均召回率为 **0.8750**；

微平均 F1 分数代表平衡精确率与召回率的矛盾，采用调和平均计算，计算公式如下：：

$$Micro - F1 = 2 \times \frac{Micro - Precision \times Micro - Recall}{Micro - Precision + Micro - Recall} \quad (63)$$

代入数据计算得，微平均 F1 分数为 **0.8750**。

结果表示微平均精确率与微平均召回率一致，且微平均 F1 分数保持相同数值，表明模型在全局样本层面的“预测纯度”与“覆盖能力”高度均衡。这种均衡性意味着模型既未因过度追求精确率而遗漏大量真正正类样本，也未因过度追求召回率而误判大量负类样本，全局分类性能稳定可靠。

宏平均指标：

宏平均指标的核心逻辑是先计算每个类别的单类指标，再对所有类别取算术平均，将每个类别视为同等重要的评估单元，适用于判断模型对小众类别的公平性，避免大众类别的性能掩盖小众类别的缺陷。计算过程如下：

1.单类指标计算：分别计算 11 类的 $Precision_i$ 、 $Recall_i$ 、 $Specificity_i$ 、 $F1_i$ ；

2.核心宏平均指标计算：对 11 类的单类指标取算术平均：

$$Macro - Precision = \frac{1}{11} \sum_{i=1}^{11} Precision_i \quad (64)$$

$$Macro - Recall = \frac{1}{11} \sum_{i=1}^{11} Recall_i \quad (65)$$

$$Macro - Specificity = \frac{1}{11} \sum_{i=1}^{11} Specificity_i \quad (66)$$

$$Macro - F1 = \frac{1}{11} \sum_{i=1}^{11} F1_i \quad (67)$$

代入数据计算得：

$Macro - Precision = 0.8745$, $Macro - Recall = 0.8887$;

$Macro - Specificity = 0.9862$, $Macro - F1 = 0.8802$;

$Macro - FPR = 0.0138$, $Macro - FNR = 0.1113$

宏平均精确率与宏平均召回率的差值仅为 0.0142，且宏平均 F1 分数达 0.8802，表明模型对 11 类的性能表现高度一致，无论是大众类别还是小众类别，均能保持稳定的精确率与召回率平衡，无明显性能偏向；宏平均特异度高达 0.9862。

本研究构建的三级指标体系具有明确的协同关系，混淆矩阵作为基础，直观呈现分类偏差，为后续指标提供数据支撑；准确率、精确率、召回率与特异度从单维度量化性能，分别解决“整体正确”“预测纯度”“覆盖能力”“负类识别”的评估需求；微平均与宏平均指标实现综合评估，前者验证全局样本均衡性，后者保障类别公平性。各指标的一致性结果共同表明：本研究的 11 类多分类模型既具备优异的整体分类性能，又兼顾不同类别的公平性需求，在实际应用中具有较强的鲁棒性与可靠性。

5.5 问题小结

本节围绕开启 TxBF 场景下 MIMO-OFDM 系统的速率估计难题，首先将六个子数据集合并，通过 IQR 方法完成时间同步异常与噪声异常的双重剔除，得到 25638 条有效样本，为适配 TxBF 的空间域信号优化特性奠定数据基础。

基于 SVD 加权改进型指数映射的等效 SINR 模型。首先基于 SVD 分解提取波束赋形权值矩阵与主奇异值，推导带子载波加权系数的改进型指数映射等效 SINR 模型，通过主奇异值占比量化不同载波的速率贡献，精准表征开启 TxBF 后的信道质量；

基于 CSI 与 SINR 多模态特征融合的信道 Transformer(MFF-CT)模型。输入层融合 CSI 原始信息、TxBF 专属的主奇异值与加权系数及 SINR 多维特征，经 Token Embedding 与位置编码映射至统一建模空间，4 层加权 Transformer 编码器通过 WMHA 机制强化对高增益子载波的关注，跨模态融合层整合序列特征与等效 SINR 全局信息，最终由速率分类层实现 11 类 mcs 索引输出。

训练结果显示，模型在 200 轮迭代后稳定收敛，验证集准确率达 87.53%，通过混淆矩阵、精确率(宏平均 0.8745)、召回率(宏平均 0.8887)及微/宏平均 F1 分数(分别为 0.8750、0.8802)验证，模型兼具全局分类均衡性与类别公平性。该模型有效适配 TxBF 的空间域信号优化机制，解决传统模型在开启 TxBF 场景下适配性差、忽略空间增益的局限，显著提升速率估计精度，为 5G 系统中依赖 TxBF 技术的远程医疗、工业控制等高可靠业务提供精准的链路速率支撑。

6. 模型评价

本文针对无线网络实际传输速率估计问题，根据无线网络传输链路容量相关物理工程背景知识，结合物理层抽象的信道质量数据，噪声数据和波束赋形数据等建立了与之匹配的 EESM 模型,基于改进型指数映射的等效 SINR 模型,基于 SVD 加权改进型指数映射的等效 SINR 模型和基于 CSI 与 SINR 多模态特征融合的信道 Transformer(MFF-CT)模型等模型，对不完备数据集 B 中提供的实际传输 MIMO 信道进行信道传输速率估计，相关工作具有一定的优越性和局限性。

6.1 模型优点

1. **数据预处理针对性强，奠定可靠建模基础**：针对数据采集时间异步问题，采用 IQR（四分位距）方法精准剔除 CSI 与传输速率采集时间差异异常值，从源头减少了因数据不匹配导致的估计偏差。针对 mcs 非整数离散分布的特性，通过统计最小差值划分区间并线性归一化至 $[0,1]$ ，完美适配 EESM 模型输出范围，解决了直接映射的技术局限。
2. **优化算法适配性优，参数估计高效稳定**：针对目标函数非凸、样本规模大的特点，选用 Adam 自适应优化算法，通过融合一阶动量（平滑更新方向）与二阶动量（动态调整学习率），实现了等效映射系数 α 与 SINR 缩放系数 β 的高效收敛。模型在 150 轮迭代后即可稳定，三个子数据集的参数拟合结果与各自硬件特性、信道环境高度匹配，验证集准确率分别达 70.97%、66.94%、61.94%，展现了算法在非凸优化场景下的优越性。
3. **模型高度吻合物理理论，宽区间适配性好**：根据实际无线传输网络工程原理建立模型，由符号错误概率理论出发，重构改进型指数映射函数，解决了传统 EESM 模型在宽 SINR 区间近似误差大的问题。通过推导反函数计算等效 SINR，使模型在不同信道质量下均能保持较高的映射精度，为速率估计提供了更准确的信道质量量化指标。
4. **多模态特征融合全面，学习机制先进**：构建“CSI 原始特征+TxBF 专属特征+SINR 多维特征”的融合输入，既保留了 CSI 的频率域与空间域细节，又纳入了 TxBF 的定向增益特性与 SINR 的全局质量信息，实现了物理层信息的最大化利用。同时，采用 4 层加权 Transformer 编码器，通过加权自注意力机制（WMHA）增强对高增益子载波的关注，契合 TxBF 技术的空间域信号优化特性；引入正弦/余弦位置编码注入子载波频率顺序信息，解决了 Transformer “置换不变”与物理序列的矛盾。跨模态融合层通过平均池化与等效 SINR 拼接，实现了序列特征与全局特征的有机整合，大幅提升了模型对复杂信道结构的建模能力。

6.2 模型缺点

1. 基于 CSI 与 SINR 多模态特征融合的信道 Transformer(MFF-CT)模型带来了较大的计算开销，特别是需要处理大规模高维数据集时。

2. 在进行模型参数拟合时，由于数据集存在样本不平衡的问题，部分参数拟合效果可以进一步加强。

7. 结论与展望

7.1 研究结论

本文围绕 MIMO-OFDM 系统不同 TxBF 状态下的链路速率精准估计难题,根据无线网络传输链路容量相关物理及工程背景知识,以实际采集数据集为支撑,针对不同场景建立模型并优化,逐步突破传统方法局限,有效提升了实际传输速率的预测准确性。

针对问题一,关闭 TxBF 场景下 EESM 模型参数拟合难、数据时间异步及 mcs 非整数离散分布的问题,本文从数据预处理、模型构建与优化到速率预测展开了系统研究。通过 IQR 方法剔除信道状态信息 (CSI) 与传输速率采集时间差异异常值,确保数据时间同步性,为参数拟合奠定可靠数据基础;基于 mcs 的离散分布特性,设计区间划分与 $[0,1]$ 线性归一化策略,适配 EESM 模型输出特性;构建含双边误差平方和与约束违背惩罚项的非凸目标函数,结合 Adam 自适应优化算法精准拟合等效映射系数 α 与 SINR 缩放系数 β 。采用分终端独立训练策略适配不同硬件与信道差异并成功检验模型泛化性,三个子数据集验证集准确率分别达 70.97%、66.94%、61.94%,成功完成数据集 B 的载波单元 SINR 计算、等效 SINR 映射与速率预测,验证了该方案在基础场景下的有效性。

针对问题二,在传统模型基础上进一步提升等效信干噪比对各子载波信干噪比特征的刻画,完成映射精度提升与特征维度扩展的系统性研究。本文基于符号错误概率理论重构改进型指数映射函数,降低宽 SINR 区间的近似误差,提升等效 SINR 表征精度;引入 MIMO 信道奇异值特征(均值、方差,表征信道能量集中度)与 SINR 统计特征(均值、方差、偏度,捕捉子载波质量差异),构建含正则化项的目标函数避免过拟合。优化后模型验证集准确率提升至 79.03%、75.97%、71.94%,较基础模型平均提升 9.03%,证明多物理层特征融合对复杂信道特性建模的关键作用。

针对问题三,开启 TxBF 后信道结构变化、子载波增益差异显著的新挑战,本文提出基于 CSI 与 SINR 多模态特征融合的信道 Transformer (MFF-CT) 模型。合并六个子数据集后,将 CSI 复数张量经 Token Embedding 与位置编码保,通过 4 层加权 Transformer 编码器强化高增益子载波关注度,最终融合等效 SINR 实现 mcs 预测。模型验证集准确率达 87.5%,较关闭 TxBF 场景的改进模型最大提升 15.56%,有效适配了 TxBF 的空间域信号优化机制。

7.2 未来展望

(1) 特征维度优化

现有模型虽融合了物理层多特征,但未充分考虑动态环境因素(如终端移动速度、遮挡变化)对信道的影响。后续可引入多普勒频移、信道时延扩展等动态特征,构建时空联合特征矩阵。

(2) 场景泛化能力提升

本文模型针对特定 MIMO-OFDM 配置 (2×4 天线、122 载波) 训练,泛化性受限于硬件参数与场景类型。未来可扩展数据集至不同天线规模(如大规模 MIMO 的 64×16 配置)、载波数量及频段 (Sub-6GHz 与毫米波),通过迁移学习与领域自适应技术,构建跨硬件、跨频段的通用速率估计模型,增强实际部署的灵活性。

(3) 模型参数形式优化

本文等效 SINR 模型中的缩放系数为一固定的值,但 mcs 的数据是离散分布,如模型参数同样离散分布且与之对应的形式,将有望更好的刻画相关特征,后续在数据集形式更丰富的情况下,可尝试拟合效果更精细的分级函数。

参考文献

- [1] Hcine M B, Bouallegue R. Analysis of uplink effective SINR in LTE networks[C]//2015International Wireless Communications and Mobile Computing Conference (IWCMC). IEEE,2015: 425-430.
- [2] 刘浩东.面向 5G 通信系统的自适应调制编码技术研究[D].西安电子科技大学,2022.DOI:10.27389/d.cnki.gxadu.2022.001353.
- [3] S. S. Tsai and A. Soong, "Effective-SNR mapping for modeling frame error rates in multiplestate channels", 3rd Generation Partnership Project 2 3GPP2, Tech. Rep. 3GPP2-C30-20030429-010,2003.
- [4] 池启康.IEEE 802.11be Wi-Fi 网络吞吐量增强的研究[J].电子技术与软件工程,2022,(04):49-53.DOI:10.20109/j.cnki.etse.2022.04.013.
- [5] 薛昀.MIMO 无线通信自适应传输理论方法研究[D].东南大学,2016.
- [6] Khan A, Khattak S. Improved link to system interfacing methodology for exponential effective SNR mapping using enhanced lookup tables[C]//17th IEEE International Multi Topic Conference 2014. IEEE, 2014: 41-45.
- [7] Tirkkonen O. Packet throughput of adaptive space-time modulation[C]// Fifth IEE International Conference on 3G Mobile Communication Technologies (3G 2004) The Premier Technical Conference for 3G and Beyond. IET, 2004.
- [8] Ghosh A.,Andrews J.G.,Chen R.,.Broadband wireless access with WiMax/802.16: current performance benchmarks and future potential[J].IEEE Communications Magazine: Articles, News, and Events of Interest to Communications Engineers.2005,43(2)
- [9] Schumacher L.,Pedersen K.I.,Kermoal J.P.,等.A stochastic MIMO radio channel model with experimental validation[J].IEEE Journal on Selected Areas in Communications.2002,20(6).

附录 A: matlab 代码

```
% 读取 Excel 文件并计算第十一列和第十二列时间差值的脚本
filename = "d0c1-bfe4-18f0_0.xlsx"; % 请确保文件在当前工作目录或提供完整路径

% 初始化错误计数器
errorCount = 0;
errorDetails = {};

try
    % 读取 Excel 文件的所有数据
    data = readtable(filename, 'ReadVariableNames', true);

    % 检查表格是否有足够的列
    if width(data) < 12
        error('The Excel file has fewer than 12 columns.');
```

end

% 提取所需列数据（第 11、12 列）
% 检查列的数据类型，如果是 cell 则转换为 double

```
if iscell(data{:, 11})
    csi_time1 = str2double(data{:, 11});
else
    csi_time1 = data{:, 11};
end

if iscell(data{:, 12})
    csi_time2 = str2double(data{:, 12});
else
    csi_time2 = data{:, 12};
end

% 计算第十一列与第十二列的时间差值的绝对值
time_diff = abs(csi_time1 - csi_time2); % 第十一列与第十二列的差值

% 创建结果表格
result_table = table(csi_time1, csi_time2, time_diff);
result_table.Properties.VariableNames = {'col11_time', 'col12_time',
'abs_difference'};

% 保存结果到新的 Excel 文件
output_filename = "time_differences_col11_col12.xlsx";
writetable(result_table, output_filename);
```

```

    fprintf("Time differences between columns 11 and 12 have been saved to %s\n",
output_filename);

% 绘制散点图
figure('Name', 'Scatter Plot of Time Differences between Column 11 and 12',
'Color', 'w', 'Position', [100, 100, 800, 600]);

% 第十一列与第十二列的差值散点图
scatter(1:length(time_diff), time_diff, 50, 'filled', ...
    'MarkerEdgeColor', [0.10 0.30 0.50], 'MarkerFaceColor', [0.10 0.30 0.50]);
set_plot_style('Scatter Plot of Time Differences between Column 11 and 12',
'Row Number', 'Time Difference', ...
    [1, length(time_diff)], [min(time_diff), max(time_diff)]);

% 绘制改进的频数分布图（柱状图+密度曲线）
figure('Name', 'Enhanced Frequency Distribution of Time Differences', 'Color',
'w', 'Position', [200, 200, 900, 700]);

% 创建双Y轴图
yyaxis left;

% 绘制柱状图
h = histogram(time_diff, 50, 'FaceColor', [0.10 0.30 0.50], 'EdgeColor',
'none', 'FaceAlpha', 0.7);
ylabel('Frequency', 'FontName', 'Times New Roman', 'FontSize', 18,
'FontWeight', 'bold');

% 计算并绘制密度曲线
yyaxis right;
[f, xi] = ksdensity(time_diff);
plot(xi, f, 'LineWidth', 3, 'Color', [0.85 0.15 0.15]);
ylabel('Density', 'FontName', 'Times New Roman', 'FontSize', 18, 'FontWeight',
'bold');

% 设置图表样式
set(gca, ...
    'FontName', 'Times New Roman', ...
    'FontSize', 18, ...
    'FontWeight', 'bold', ...
    'LineWidth', 2, ...
    'GridLineStyle', '--', ...
    'XGrid', 'on', ...
    'YGrid', 'on', ...
    'Box', 'on');

```

```

xlabel('Time Difference', 'FontName', 'Times New Roman', 'FontSize', 18,
'FontWeight', 'bold');

% 添加图例
legend('Frequency', 'Density Curve', 'Location', 'northwest');

% 使用 IQR 方法计算阈值
Q1 = prctile(time_diff, 25);
Q3 = prctile(time_diff, 75);
IQR = Q3 - Q1;
threshold = Q3 + 1.5 * IQR; % 常用异常值检测阈值

% 输出阈值信息
fprintf('Calculated threshold using IQR method: %.6f\n', threshold);

% 识别并移除异常值
outliers = time_diff > threshold;
clean_data = data(~outliers, :);

% 输出处理前后的数据量
fprintf('Original data size: %d rows\n', height(data));
fprintf('Data after removing outliers: %d rows\n', height(clean_data));
fprintf('Number of outliers removed: %d rows\n', sum(outliers));

% 保存处理后的数据
clean_output_filename = "cleaned_data.xlsx";
writetable(clean_data, clean_output_filename);
fprintf("Cleaned data has been saved to %s\n", clean_output_filename);

catch ME
    errorCount = errorCount + 1;
    errorDetails{errorCount} = ME.message;
    fprintf("An error occurred: %s\n", ME.message);
end

% 绘图风格函数
function set_plot_style(title_str, x_label, y_label, x_lim, y_lim)
    % 坐标轴设置
    set(gca, ...
        'FontName', 'Times New Roman', ...
        'FontSize', 18, ...
        'FontWeight', 'bold', ...
        'LineWidth', 2, ...

```

```

        'GridLineStyle', '--', ...
        'XGrid', 'on', ...
        'YGrid', 'on', ...
        'Box', 'on');

    if ~isempty(x_lim)
        xlim(x_lim);
    end
    if ~isempty(y_lim)
        ylim(y_lim);
    end

    xlabel(x_label, 'FontName', 'Times New Roman', 'FontSize', 18, 'FontWeight',
'bold');
    ylabel(y_label, 'FontName', 'Times New Roman', 'FontSize', 18, 'FontWeight',
'bold');
end

```

```

numh = height(sheet1)
for layer = 1:numh
% 获取当前行的所有单元格
currentRow = sheet1(layer, :);
% 循环处理每个单元格
for row = 1:size(currentRow, 2)
% 跳过空单元格
if isempty(currentRow{row})
continue;
end
% 分割字符串得到复数文本数组
complexStrs = strsplit(currentRow{row}, ',');
% 确保每个单元格有 122 个复数
if length(complexStrs) ~= 122
warning('第%d 行第%d 个单元格包含%d 个复数, 不是 122 个', layer, row,
length(complexStrs));
continue;
end
% 将每个复数文本转换为复数并存储到张量
for i = 1:122
% 转换字符串到复数
q3tensor1(row, i, layer) = str2double(complexStrs{i});
end
end

```

```
end
```

```
function [result, SINR] = process_tensor(tensor3D, noise, row)

if size(tensor3D) ~= [8, 122, row]
error('输入张量维度应为 8×122×%d', row);
end
% 验证噪声向量长度
if length(noise) ~= row
error('噪声向量长度应等于%d', row);
end
% 初始化结果数组
result = zeros(122, row);
SINR = zeros(122, row);
% 循环计算每 8 个复数模的平方和及 SINR
for k = 1:row
% 计算噪声功率
noisepower = 10^((noise(k) - 30) / 10);
for j = 1:122
% 提取当前位置的 8 个复数
current_values = tensor3D(:, j, k);
% 计算每个复数模的平方并求和
result(j, k) = sum(abs(current_values).^2);
% 计算 SINR
SINR(j, k) = result(j, k) / Iacc;
end
end
end
```

```

timedata = abs(q1var3(:,2)-q1var3(:,3));
q1delete3 = detect_and_report_outliers(timedata, q1var3(:,4));

function outlier_indices = detect_and_report_outliers(time, noise)
if length(time) ~= length(noise)
error('time 和 noise 的长度必须相同');
end
if ~isvector(time) || ~isvector(noise)
error('time 和 noise 必须是向量');
end
% 转换为列向量
time = time(:);
noise = noise(:);
total_samples = length(time);
% 计算 time 的四分位数和正常范围
q1_time = quantile(time, 0.25);
q3_time = quantile(time, 0.75);
iqr_time = q3_time - q1_time;
time_lower = q1_time - 1.5 * iqr_time;
time_upper = q3_time + 1.5 * iqr_time;
% 计算 noise 的四分位数和正常范围
q1_noise = quantile(noise, 0.25);
q3_noise = quantile(noise, 0.75);
iqr_noise = q3_noise - q1_noise;
noise_lower = q1_noise - 1.5 * iqr_noise;
noise_upper = q3_noise + 1.5 * iqr_noise;
% 检测异常值
time_outlier = (time < time_lower) | (time > time_upper);
noise_outlier = (noise < noise_lower) | (noise > noise_upper);
% 获取异常值索引
time_indices = find(time_outlier);
noise_indices = find(noise_outlier);
outlier_indices = unique([time_indices; noise_indices]); % 去重后的索引
% 计算异常值数量
time_outlier_count = length(time_indices);
noise_outlier_count = length(noise_indices);
total_outlier_count = length(outlier_indices);

[alpha_opt, beta_opt] = main_optimization(dbSINR, boundnono, 20,500000)

q1combined1 = cat(2,q1string3,q1var3(:,1),q1var3(:,4));
poq3 = delete_elements(q1combined1, 1, q1delete3);
function result = delete_elements(original_array, dim, delete_indices)

```

```

if dim < 1 || dim > ndims(original_array)
error('指定的维度超出数组维度范围');
end
% 将删除索引转换为行向量并去重、排序
delete_indices = unique(sort(delete_indices(:).')); % 确保索引唯一且有序
% 检查索引是否在有效范围内
dim_size = size(original_array, dim);
if any(delete_indices < 1) || any(delete_indices > dim_size)
error('删除索引超出维度范围');
end
% 计算需要保留的索引
all_indices = 1:dim_size;
keep_indices = setdiff(all_indices, delete_indices);
% 构造索引单元格数组，用于访问数组
indices = cell(1, ndims(original_array));
for i = 1:ndims(original_array)
if i == dim
indices{i} = keep_indices; % 在指定维度使用保留的索引
else
indices{i} = ':'; % 其他维度保留所有元素
end
end
% 执行删除操作
result = original_array(indices{:});
end

```

```

function [alpha_opt, beta_opt] = main_optimization(sinr_data, bound, alpha_init,
beta_init)
if nargin < 3
alpha_init = 1;
beta_init = 1;
end
% 初始参数
x0 = [alpha_init; beta_init];
% 创建优化选项
options = optimoptions('fminunc', ...
'Algorithm', 'quasi-newton', ...
'HessUpdate', 'lbfgs', ...
'Display', 'iter', ...
'MaxIterations', 100, ...
'MaxFunctionEvaluations', 1000000, ... % 增加此参数值
'OptimalityTolerance', 1e-6, ...

```

```

'OutputFcn', @(x, optimValues, state) outfun(x, optimValues, state, sinr_data,
bound));
% 创建匿名函数, 传递 sinr_data 和 bound
objective = @(x) loss_function(x, sinr_data, bound);
% 运行优化
[x_opt, fval] = fminunc(objective, x0, options);
% 提取优化结果
alpha_opt = x_opt(1);
beta_opt = x_opt(2);
% 显示最终结果
% 显示最终结果
fprintf('优化完成! \n');
fprintf('最优 alpha 值: %.4f\n', alpha_opt);
fprintf('最优 beta 值: %.4f\n', beta_opt);
fprintf('最终损失值: %.4f\n', fval);
% 可视化最终结果
visualize_results(alpha_opt, beta_opt, sinr_data, bound);
end

function stop = outfun(x, optimValues, state, sinr_data, bound)
% 输出函数, 用于可视化优化过程
persistent history;
stop = false;
switch state
case 'init'
% 初始化历史记录
history.fval = [];
history.x = [];
history.iter = [];
% 创建图形窗口
figure('Name', '优化过程可视化', 'Position', [100, 100, 1000, 600]);
case 'iter'
% 记录历史
history.fval = [history.fval; optimValues.fval];
history.x = [history.x; x'];
history.iter = [history.iter; optimValues.iteration];
% 更新图形
subplot(2, 2, 1);
plot(history.iter, history.fval, 'b-o');
title('损失函数值');
xlabel('迭代次数');
ylabel('损失值');
grid on;
subplot(2, 2, 2);

```

```

plot(history.iter, history.x(:,1), 'r-o');
title('Alpha 值变化');
xlabel('迭代次数');
ylabel('Alpha');
grid on;
subplot(2, 2, 3);
plot(history.iter, history.x(:,2), 'g-o');
title('Beta 值变化');
xlabel('迭代次数');
ylabel('Beta');
grid on;
% 计算当前参数下的预测值
alpha_current = x(1);
beta_current = x(2);
mcs_pred = model_fun(alpha_current, beta_current, sinr_data);
% 绘制预测值与参考范围的对比
subplot(2, 2, 4);
hold off;
% 绘制参考范围
n_samples = size(bound, 1);
x_indices = 1:n_samples;
% 绘制参考范围区域
x_area = [x_indices, fliplr(x_indices)];
y_area = [bound(:,1)', fliplr(bound(:,2)')];
fill(x_area, y_area, [0.8, 0.8, 1], 'EdgeColor', 'none', 'FaceAlpha', 0.3);
hold on;
% 绘制预测值
plot(x_indices, mcs_pred, 'r.-', 'LineWidth', 1.5);
% 绘制上下界
plot(x_indices, bound(:,1), 'b--', 'LineWidth', 1);
plot(x_indices, bound(:,2), 'b--', 'LineWidth', 1);
title('预测值与参考范围对比');
xlabel('样本索引');
ylabel('值');
legend('参考范围', '预测值', '下界', '上界');
grid on;
% 计算在范围内的样本比例
in_range = sum(mcs_pred >= bound(:,1) & mcs_pred <= bound(:,2));
in_range_percent = in_range / n_samples * 100;
% 更新标题显示在范围内的比例
title(sprintf('预测值与参考范围对比 (在范围内: %.1f%%)', in_range_percent));
drawnow;
case 'done'
% 优化完成

```

```

end
end

function f = loss_function(x, sinr_data, bound)
alpha = x(1);
beta = x(2);
% 计算模型预测值
mcs_pred = model_fun(alpha, beta, sinr_data);
% 计算与边界的平方差
lower_bound = bound(:,1);
upper_bound = bound(:,2);
% 初始化损失
loss = 0;
% 对每个样本计算损失
for i = 1:length(mcs_pred)
pred = mcs_pred(i);
lb = lower_bound(i);
ub = upper_bound(i);
% 如果预测值在范围内，不施加惩罚
if pred >= lb && pred <= ub
% 不添加惩罚项
else
% 如果预测值小于下界，计算与下界的平方差
if pred < lb
loss = loss + (pred - lb)^2;
% 如果预测值大于上界，计算与上界的平方差
elseif pred > ub
loss = loss + (pred - ub)^2;
end
% 添加额外惩罚项
penalty = 10; % 惩罚系数，可调整
loss = loss + penalty;
end
end
% 添加参数正则化项，防止参数过大
lambda = 0.01; % 正则化系数，可调整
reg_term = lambda * (alpha^2 + beta^2);
% 总损失
f = loss + reg_term;
end

function mcs_pred = model_fun(alpha, beta, sinr_data)
% 目标函数

```

```

phi = @(x) 1 - exp(-x); % 假设  $\Phi(x) = 1 - \exp(-x)$ 
[~, n_samples] = size(sinr_data);
mcs_pred = zeros(n_samples, 1); % 初始化 n_samples×1 列向量
for i = 1:n_samples
    sinr_row = sinr_data(:, i);
    normalized = sinr_row / beta;
    phi_vals = phi(normalized);
    avg_phi = mean(phi_vals);
    mcs_pred(i) = alpha * avg_phi;
end
end

function visualize_results(alpha, beta, sinr_data, bound)
% 可视化最终结果
% 计算模型预测值
mcs_pred = model_fun(alpha, beta, sinr_data);
% 创建新图形
figure('Name', '最终优化结果', 'Position', [200, 200, 800, 600]);
% 绘制预测值与参考范围的对比
n_samples = size(bound, 1);
x_indices = 1:n_samples;
% 绘制参考范围区域
x_area = [x_indices, fliplr(x_indices)];
y_area = [bound(:,1)', fliplr(bound(:,2)')];
fill(x_area, y_area, [0.8, 0.8, 1], 'EdgeColor', 'none', 'FaceAlpha', 0.3);
hold on;
% 绘制预测值
plot(x_indices, mcs_pred, 'r.-', 'LineWidth', 1.5);
% 绘制上下界
plot(x_indices, bound(:,1), 'b--', 'LineWidth', 1);
plot(x_indices, bound(:,2), 'b--', 'LineWidth', 1);
% 计算在范围内的样本比例
in_range = sum(mcs_pred >= bound(:,1) & mcs_pred <= bound(:,2));
in_range_percent = in_range / n_samples * 100;
title(sprintf('最终预测值与参考范围对比 (在范围内: %.1f%%)', in_range_percent));
xlabel('样本索引');
ylabel('值');
legend('参考范围', '预测值', '下界', '上界');
grid on;
% 添加文本显示最优参数
text_str = sprintf('最优参数: \alpha = %.4f, \beta = %.4f', alpha, beta);
annotation('textbox', [0.15, 0.02, 0.7, 0.05], 'String', text_str, ...
'EdgeColor', 'none', 'HorizontalAlignment', 'center', ...
'FontSize', 12, 'FontWeight', 'bold');

```

```
end
```

```
% 统计 6 个矩阵第一列中相同元素的个数
count_common_elements(q3var1, q3var2, q3var3, q3var4, q3var5, q3var6)
function count_common_elements(matrix1, matrix2, matrix3, matrix4, matrix5,
matrix6)
% 提取每个矩阵的第一列
col1 = matrix1(:, 1);
col2 = matrix2(:, 1);
col3 = matrix3(:, 1);
col4 = matrix4(:, 1);
col5 = matrix5(:, 1);
col6 = matrix6(:, 1);
% 合并所有第一列的元素
all_elements = [col1; col2; col3; col4; col5; col6];
% 找出唯一元素并统计每个元素的出现次数
[unique_elements, ~, element_indices] = unique(all_elements);
element_counts = accumarray(element_indices, 1);
% 显示结果
result_table = table(unique_elements, element_counts, ...
'VariableNames', {'元素值', '出现次数'});
disp('6 个矩阵第一列中各元素的出现次数: ');
disp(result_table);
% 如果需要统计在所有 6 个矩阵中都出现过的元素
fprintf('\n 在所有 6 个矩阵第一列中都出现过的元素: \n');
common_elements = [];
for i = 1:length(unique_elements)
    elem = unique_elements(i);
% 检查元素是否在每个矩阵的第一列中都存在
in1 = any(col1 == elem);
in2 = any(col2 == elem);
in3 = any(col3 == elem);
in4 = any(col4 == elem);
```

```

in5 = any(col5 == elem);
in6 = any(col6 == elem);
if in1 && in2 && in3 && in4 && in5 && in6
common_elements = [common_elements; elem];
end
end
if ~isempty(common_elements)
disp(common_elements);
fprintf('共有 %d 个元素在所有 6 个矩阵的第一列中都出现过\n', length(common_elements));
else
disp('没有元素在所有 6 个矩阵的第一列中都出现过');
end
end

function [sinr_matrix, noise_power] = calculate_txbf_sinr_dbm(csi_tensor,
noise_dBm)
[num_h, num_carriers, num_samples] = size(csi_tensor);
% 验证信道张量维度（必须为 8×122×nsample，对应 2Rx×4Tx）
if num_h ~= 8 || num_carriers ~= 122
error('信道张量维度错误！需为 8×122×nsample（8=2Rx×4Tx，122 子载波）');
end
% 验证底噪向量长度（需与样本数一致）
if length(noise_dBm) ~= num_samples
error('底噪向量长度（%d）与样本数（%d）不匹配', length(noise_dBm), num_samples);
end

% 初始化输出
sinr_matrix = zeros(num_carriers, num_samples); % 122×nsample SINR 矩阵
noise_power = zeros(num_samples, 1); % 转换后的底噪（单位：W）

for s = 1:num_samples
dbm = noise_dBm(s);
% 1. dBm → mW: P_mW = 10^(dBm/10)
p_mw = 10^(dbm / 10);
% 2. mW → W: 1 W = 1000 mW
p_w = p_mw / 1000;
noise_power(s) = max(p_w, 1e-16);

for sample_idx = 1:num_samples
% 提取当前样本的所有子载波信道数据（8×122）

```

```

csi_sample = csi_tensor(:, :, sample_idx);
% 提取当前样本的线性底噪功率 (W)
current_noise = noise_power(sample_idx);

% 遍历每个子载波 (122 个)
for carrier_idx = 1:num_carriers
% 从 8x1 向量中提取当前子载波的信道系数, 重构为 2Rx×4Tx 矩阵
h_vec = csi_sample(:, carrier_idx); % 8 个信道系数 (按 Rx-Rx-Tx 顺序存储)
H = reshape(h_vec, 2, 4); % 2x4 复信道矩阵: H(n,m) = 第 m 根 Tx 到第 n 根 Rx 的系数

H_conjT = H'; % 共轭转置 (复矩阵需用', 实矩阵等价于转置)
norm_H = norm(H_conjT, 'fro'); % Frobenius 范数 (矩阵元素平方和开根号)
V = H_conjT / norm_H; % 4x2 权重矩阵 (4Tx×2 流, 流数=接收天线数)

signal_power = norm(H * V, 'fro')^2; % ||H*V||_F^2 (Frobenius 范数平方)
% 2. SINR (线性域转 dB): SINR_dB = 10*log10(信号功率/噪声功率)
sinr_linear = signal_power / current_noise;
sinr_matrix(carrier_idx, sample_idx) = 10 * log10(sinr_linear);
end

% 显示计算进度 (每 10 个样本打印一次)
if mod(sample_idx, 10) == 0
fprintf('已处理 %d/%d 个样本\n', sample_idx, num_samples);
end
end
end

```

```

function noise_power = dbm2watt(noise_dBm)
p_mw = 10.^(noise_dBm / 10);
noise_power = p_mw / 1000;
noise_power = max(noise_power, 1e-16); % 避免零功率
end

function [sinri_122, sinreff_dB] = eESM_calculation(csi_tensor, noise_dBm)
[num_h, num_carriers, num_samples] = size(csi_tensor);
if num_h ~= 8 || num_carriers ~= 122
error('输入张量必须为 8x122xnsample (2Rx×4Tx, 122 子载波)');
end

```

```

% 初始化输出
sinri_122 = zeros(num_samples, num_carriers); % 每个样本 122 个 SINR
sinreff_dB = zeros(num_samples, 1); % 每个样本 1 个等效值
% 底噪转换
noise_power = dbm2watt(noise_dBm);
% 遍历每个样本
for sample_idx = 1:num_samples
% 提取当前样本的信道数据和底噪
csi_sample = csi_tensor(:, :, sample_idx); % 8×122 信道
current_noise = noise_power(sample_idx); % 底噪功率 (W)
for carrier_idx = 1:122
% 1.1 重构 2×4 信道矩阵 H
h_vec = csi_sample(:, carrier_idx); % 8 个信道系数

```

```

function calculateClassificationMetrics(confMat, classLabels)
if nargin < 2 || isempty(classLabels)
classLabels = cell(11,1);
for i = 1:11
classLabels{i} = sprintf('类别%d', i);
end
else
if length(classLabels) ~= 11
error('类别标签数量必须为 11 个');
end
end
numClasses = 11; % 类别数
totalSamples = sum(confMat(:)); % 总样本数
correctSamples = sum(diag(confMat)); % 所有正确分类的样本数
%% 1. 计算每类的基础指标 (TP、FP、FN、TN)
TP = diag(confMat); % 真正例: 真实为 i, 预测为 i
FP = sum(confMat,1)' - TP; % 假正例: 真实不为 i, 预测为 i (列和 - TP)
FN = sum(confMat,2) - TP; % 假负例: 真实为 i, 预测不为 i (行和 - TP)
TN = totalSamples - TP - FP - FN; % 真负例: 真实不为 i, 预测不为 i
%% 2. 计算每类的局部指标
Precision = TP ./ (TP + FP + eps); % 精确率
Recall = TP ./ (TP + FN + eps); % 召回率
Specificity = TN ./ (TN + FP + eps); % 特异度
F1 = 2 * (Precision .* Recall) ./ (Precision + Recall + eps); % F1 分数
FPR = FP ./ (FP + TN + eps); % 假正例率
FNR = FN ./ (TP + FN + eps); % 假负例率
ClassAccuracy = (TP + TN) ./ totalSamples; % 每类准确率

```

```

%% 3. 计算全局指标
% 总体准确率
overallAccuracy = correctSamples / totalSamples;
% Micro-averaged 指标（先汇总再计算）
micro_TP = sum(TP);
micro_FP = sum(FP);
micro_FN = sum(FN);
micro_Precision = micro_TP / (micro_TP + micro_FP + eps);
micro_Recall = micro_TP / (micro_TP + micro_FN + eps);
micro_F1 = 2 * (micro_Precision * micro_Recall) / (micro_Precision + micro_Recall + eps);
% Macro-averaged 指标（先计算每类再平均）
macro_Precision = mean(Precision);
macro_Recall = mean(Recall);
macro_Specificity = mean(Specificity);
macro_F1 = mean(F1);
macro_FPR = mean(FPR);
macro_FNR = mean(FNR);
%% 4. 打印所有指标
% 打印局部指标
fprintf('=== 局部指标（每类详细结果） ===\n');
fprintf('%-10s %-6s %-6s %-6s %-6s %-10s %-10s %-10s %-10s %-10s %-10s\n', ...
    '类别', 'TP', 'FP', 'FN', 'TN', 'Precision', 'Recall', 'Specificity', ...
    'F1', 'FPR', 'FNR', 'Accuracy');
fprintf('-----\n');

for k = 1:numClasses
    fprintf('%-10s %-6d %-6d %-6d %-6d %-10.4f %-10.4f %-10.4f %-10.4f %-10.4f %-10.4f\n', ...
        classLabels{k}, TP(k), FP(k), FN(k), TN(k), ...
        Precision(k), Recall(k), Specificity(k), ...
        F1(k), FPR(k), FNR(k), ClassAccuracy(k));
end
% 打印全局指标
fprintf('\n\n=== 全局指标 ===\n');
fprintf('总体准确率（Overall Accuracy）: %.4f\n', overallAccuracy);
fprintf('\n--- Micro-averaged 指标 ---\n');
fprintf('Precision: %.4f, Recall: %.4f, F1: %.4f\n', ...
    micro_Precision, micro_Recall, micro_F1);
fprintf('\n--- Macro-averaged 指标 ---\n');
fprintf('Precision: %.4f, Recall: %.4f, Specificity: %.4f\n', ...
    macro_Precision, macro_Recall, macro_Specificity);
fprintf('F1: %.4f, FPR: %.4f, FNR: %.4f\n', ...

```

```

macro_F1, macro_FPR, macro_FNR);
pinkNodes = [
1.00, 1.00, 1.00; % 纯白（低值）
0.90, 0.80, 0.85; % 粉灰
0.80, 0.50, 0.65;
0.8294, 0.3412, 0.4118]; % 深粉色（高值）
% 插值生成 256 色平滑渐变
pinkColormap = interp1(linspace(0,1,4), pinkNodes, linspace(0,1,256), 'linear');

%% 5. 混淆矩阵可视化（带类别标签）
figure('Name', '11×11 混淆矩阵热力图', 'Position', [100 100 900 700]);
heatmap(classLabels, classLabels, confMat, ... % 使用类别标签作为坐标轴
'Colormap', pinkColormap, ...
'ColorbarVisible', 'on', ...
'XLabel', '预测类别', ...
'YLabel', '真实类别', ...
'Title', '11 分类模型混淆矩阵热力图');
% 调整字体大小以确保标签清晰显示
title('11 分类模型混淆矩阵热力图', 'FontSize', 14);
xlabel('预测类别', 'FontSize', 12);
ylabel('真实类别', 'FontSize', 12);
end

H = reshape(h_vec, 2, 4); % 2Rx4Tx 矩阵
% 1.2 TxBF 权重（MRT 准则）：  $V = H^H H / ||H^H H||_F$ 
H_H = H'; % 共轭转置
norm_H = norm(H_H, 'fro');
V = H_H / max(norm_H, 1e-10); % 归一化，避免除零
% 1.3 信号功率：  $||H*V||^2_F$ 
signal_power = norm(H * V, 'fro')^2;
% 1.4 子载波 SINR (dB)
sinri_122(sample_idx, carrier_idx) = 10*log10(signal_power / current_noise);
end
sinr_dB = sinri_122(sample_idx, :); % 1×122 向量（当前样本的所有子载波 SINR）
% 公式拆解：
exp_terms = exp(-2.65 * sqrt(sinr_dB) - 0.113);
avg_exp = mean(exp_terms); % mean()等价于 sum()/122
sinr_eff_linear = (-0.377 * log(avg_exp) - 0.0426)^2;
sinreff_dB(sample_idx) = 10*log10(sinr_eff_linear);
% 进度显示
end

```

附录 B: Python 代码

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from tqdm import tqdm

# --- [PART 1: 数据加载 (已重写以支持复数)] ---
def load_and_preprocess_data_for_classification(filename):
    print(f'Step 1: Loading and preprocessing COMPLEX data from '{filename}'...')
    try:
        df = pd.read_excel(filename)
    except FileNotFoundError:
        print(f'错误: 找不到文件 '{filename}'。");
        return None, None, None, None, None
    except ImportError:
        print("错误: 缺少 'openpyxl' 库。请运行 'pip install openpyxl'。");
        return None, None, None, None, None

# --- 1. 解析复数特征 X ---
def parse_complex_cell(cell_str):
    """解析包含复数字符串的单元格"""
    try:
        # 去除多余空格, 并将 'i' 替换为 'j' 以符合 Python 标准
        s_cleaned = str(cell_str).replace(' ', '').replace('i', 'j')
        # 按逗号分割成单个复数字符串
        complex_strings = s_cleaned.split(',')
        # 将字符串列表转换为 numpy 复数数组
        return np.array([np.complex128(c) for c in complex_strings])
    except (ValueError, TypeError):
        # 如果解析失败, 返回一个全零的复数数组
        return np.zeros(122, dtype=np.complex128)

feature_columns = df.columns[:8]
X_complex_list = []
print("Parsing complex numbers from cells...")
```

```

for index, row in tqdm(df.iterrows(), total=df.shape[0], desc="Processing rows"):
    row_features = [parse_complex_cell(row[col]) for col in feature_columns]
    X_complex_list.append(row_features)

# 形状: (样本数, 8, 122) 的复数数组
X_complex = np.array(X_complex_list)

# --- 2. 核心改动: 将复数拆分为 实部-虚部 双通道 ---
# 提取实部和虚部
X_real = X_complex.real.astype(np.float32)
X_imag = X_complex.imag.astype(np.float32)

# 将实部和虚部堆叠成两个通道。形状: (样本数, 2, 8, 122)
X_two_channel = np.stack([X_real, X_imag], axis=1)

# --- 3. 标准化特征 (分别对实部和虚部进行) ---
# 为了正确标准化, 我们需要对每个通道单独进行
scaler_real = StandardScaler()
scaler_imag = StandardScaler()

# 展平数据以进行标准化
X_real_flat = X_real.reshape(-1, X_real.shape[-1])
X_imag_flat = X_imag.reshape(-1, X_imag.shape[-1])

# 标准化
X_real_scaled_flat = scaler_real.fit_transform(X_real_flat)
X_imag_scaled_flat = scaler_imag.fit_transform(X_imag_flat)

# 恢复形状
X_real_scaled = X_real_scaled_flat.reshape(X_real.shape)
X_imag_scaled = X_imag_scaled_flat.reshape(X_imag.shape)

# 再次堆叠成最终输入
X_final = np.stack([X_real_scaled, X_imag_scaled], axis=1)

# --- 4. 处理目标 Y 和类别权重 (与之前相同) ---
target_column = df.columns[8]
Y_raw = df[target_column].values
unique_classes = np.unique(Y_raw)
num_classes = len(unique_classes)
class_weights = compute_class_weight('balanced', classes=unique_classes, y=Y_raw)
class_weights_tensor = torch.tensor(class_weights, dtype=torch.float)
value_to_idx = {v: i for i, v in enumerate(unique_classes)}
idx_to_value = {i: v for i, v in enumerate(unique_classes)}

```

```

Y_labels = np.array([value_to_idx[v] for v in Y_raw], dtype=np.int64)

print(f"\nFinal input shape (X) with 2 channels (Real, Imag): {X_final.shape}")

return X_final, Y_labels, num_classes, idx_to_value, class_weights_tensor

# --- [PART 2: 模型定义 (输入通道已更新)] ---
class ResidualBlock(nn.Module): # (残差块不变)
    # ...
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResidualBlock, self).__init__()
        self.relu = nn.ReLU()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1,
bias=False);
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1, bias=False);
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.downsample = nn.Sequential()
        if stride != 1 or in_channels != out_channels: self.downsample = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, bias=False),
            nn.BatchNorm2d(out_channels))

    def forward(self, x):
        identity = self.downsample(x);
        out = self.relu(self.bn1(self.conv1(x)));
        out = self.bn2(self.conv2(out));
        out += identity;
        out = self.relu(out)
        return out

class PhysicallyInformedClassifier(nn.Module):
    def __init__(self, num_classes, dropout_rate=0.5):
        super(PhysicallyInformedClassifier, self).__init__()
        self.in_channels = 32
        self.relu = nn.ReLU()

    # --- 核心改动: 输入通道数从 1 变为 2 ---
    self.freq_feature_extractor = nn.Sequential(
        nn.Conv2d(2, 32, kernel_size=(1, 11), stride=1, padding=(0, 5), bias=False),
        nn.BatchNorm2d(32),
        nn.ReLU()
    )

```

```

# ... (后续层定义不变) ...
self.layer1 = self._make_layer(ResidualBlock, 64, 2, stride=2)
self.layer2 = self._make_layer(ResidualBlock, 128, 2, stride=2)
self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
self.dropout = nn.Dropout(p=dropout_rate)
self.fc = nn.Linear(128, num_classes)
self._initialize_weights()

# ... (其他方法 _make_layer, forward, _initialize_weights 不变) ...
def _make_layer(self, block, out_channels, num_blocks, stride):
    strides = [stride] + [1] * (num_blocks - 1);
    layers = []
    for s in strides: layers.append(block(self.in_channels, out_channels, s)); self.in_channels = out_channels
    return nn.Sequential(*layers)

def forward(self, x):
    out = self.freq_feature_extractor(x);
    self.in_channels = 32;
    out = self.layer1(out);
    out = self.layer2(out)
    out = self.avgpool(out);
    out = torch.flatten(out, 1);
    out = self.dropout(out);
    out = self.fc(out)
    return out

def _initialize_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, 1); nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Linear):
            nn.init.xavier_normal_(m.weight)
            if m.bias is not None: nn.init.constant_(m.bias, 0)

# --- [PART 3: 训练与评估 (完全不变)] ---
def train_and_classify(data_file):
    res = load_and_preprocess_data_for_classification(data_file)
    if res[0] is None: return
    X, Y, num_classes, idx_to_value, class_weights = res
    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42, stratify=Y)

```

```

train_dataset = TensorDataset(torch.from_numpy(X_train), torch.from_numpy(Y_train));
test_dataset = TensorDataset(torch.from_numpy(X_test), torch.from_numpy(Y_test))
BATCH_SIZE = 32;
train_loader = DataLoader(dataset=train_dataset, batch_size=BATCH_SIZE, shuffle=True);
test_loader = DataLoader(dataset=test_dataset, batch_size=BATCH_SIZE, shuffle=False)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu');
print(f"\nUsing device: {device}")
model = PhysicallyInformedClassifier(num_classes=num_classes).to(device)
class_weights = class_weights.to(device)
criterion = nn.CrossEntropyLoss(weight=class_weights)
optimizer = torch.optim.Adam(model.parameters(), lr=5e-4, weight_decay=1e-4)
NUM_EPOCHS = 80
print("\nStarting model training with COMPLEX-AWARE (2-channel) architecture...")
# ... (训练和评估循环完全不变) ...
for epoch in range(NUM_EPOCHS):
    model.train()
    progress_bar = tqdm(train_loader, desc=f'Epoch {epoch + 1}/{NUM_EPOCHS}', leave=False)
    for inputs, labels in progress_bar:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs);
        loss = criterion(outputs, labels);
        optimizer.zero_grad();
        loss.backward();
        optimizer.step()
        progress_bar.set_postfix(loss=loss.item())
print("\nTraining finished!")
model.eval();
all_predictions, all_actuals = [], []
with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs);
        _, predicted_indices = torch.max(outputs.data, 1)
        all_predictions.extend(predicted_indices.cpu().numpy());
        all_actuals.extend(labels.cpu().numpy())
accuracy = accuracy_score(all_actuals, all_predictions)
print(f"\n--- Final Results on '{data_file}' ---");
print(f"Final Accuracy on Test Set: {accuracy * 100:.2f}%")
actual_values = [idx_to_value[idx] for idx in all_actuals];
predicted_values = [idx_to_value[idx] for idx in all_predictions]
class_names = [str(idx_to_value[i]) for i in sorted(idx_to_value.keys())]
print("\nDetailed Classification Report:");
print(classification_report(actual_values, predicted_values, target_names=class_names, zero_division=0))

```

```

if __name__ == '__main__':
    full_data_filename = "1.xlsx"
    train_and_classify(full_data_filename)

```

附录 C：部分计算结果展示

问题 1 中 1)每个载波单元的 SINR 计算 2)将所有载波单元的 SINR 进行等效 SINR 映射的部分结果(dB)

样本 序号	SINR1	SINR2	SINR3	SINR4	SINR5	SINR6	SINReff
1	42.03	42.03	42.03	42.03	42.03	42.03	43.04
2	34.03	34.03	34.03	34.03	34.03	34.03	34.85
3	35.78	35.78	35.78	35.78	35.78	35.78	36.64
4	36.28	36.28	36.28	36.28	36.28	36.28	37.15
5	38.53	38.53	38.53	38.53	38.53	38.53	39.45
6	38.03	38.03	38.03	38.03	38.03	38.03	38.94
7	34.28	34.28	34.28	34.28	34.28	34.28	35.10
8	38.03	38.03	38.03	38.03	38.03	38.03	38.94
9	36.53	36.53	36.53	36.53	36.53	36.53	37.41
10	35.78	35.78	35.78	35.78	35.78	35.78	36.64
11	36.03	36.03	36.03	36.03	36.03	36.03	36.89
12	36.03	36.03	36.03	36.03	36.03	36.03	36.89
13	38.03	38.03	38.03	38.03	38.03	38.03	38.94
14	38.28	38.28	38.28	38.28	38.28	38.28	39.20
15	38.03	38.03	38.03	38.03	38.03	38.03	38.94
16	38.03	38.03	38.03	38.03	38.03	38.03	38.94
17	36.03	36.03	36.03	36.03	36.03	36.03	36.89
18	38.03	38.03	38.03	38.03	38.03	38.03	38.94
19	38.03	38.03	38.03	38.03	38.03	38.03	38.94
20	37.53	37.53	37.53	37.53	37.53	37.53	38.43
21	38.53	38.53	38.53	38.53	38.53	38.53	39.45
22	38.03	38.03	38.03	38.03	38.03	38.03	38.94
23	37.28	37.28	37.28	37.28	37.28	37.28	38.17
24	38.03	38.03	38.03	38.03	38.03	38.03	38.94
25	38.03	38.03	38.03	38.03	38.03	38.03	38.94
26	34.28	34.28	34.28	34.28	34.28	34.28	35.10

27	36.03	36.03	36.03	36.03	36.03	36.03	36.89
28	35.78	35.78	35.78	35.78	35.78	35.78	36.64
29	38.53	38.53	38.53	38.53	38.53	38.53	39.45
30	36.03	36.03	36.03	36.03	36.03	36.03	36.89
31	36.03	36.03	36.03	36.03	36.03	36.03	36.89
32	38.03	38.03	38.03	38.03	38.03	38.03	38.94
33	36.28	36.28	36.28	36.28	36.28	36.28	37.15
34	42.53	42.53	42.53	42.53	42.53	42.53	43.55
35	35.78	35.78	35.78	35.78	35.78	35.78	36.64
36	33.78	33.78	33.78	33.78	33.78	33.78	34.59
37	36.03	36.03	36.03	36.03	36.03	36.03	36.89
38	38.03	38.03	38.03	38.03	38.03	38.03	38.94
39	37.53	37.53	37.53	37.53	37.53	37.53	38.43
40	38.03	38.03	38.03	38.03	38.03	38.03	38.94
41	38.03	38.03	38.03	38.03	38.03	38.03	38.94
42	38.28	38.28	38.28	38.28	38.28	38.28	39.20
43	38.03	38.03	38.03	38.03	38.03	38.03	38.94
44	38.03	38.03	38.03	38.03	38.03	38.03	38.94
45	38.03	38.03	38.03	38.03	38.03	38.03	38.94
46	37.53	37.53	37.53	37.53	37.53	37.53	38.43
47	38.28	38.28	38.28	38.28	38.28	38.28	39.20
48	38.03	38.03	38.03	38.03	38.03	38.03	38.94
49	37.53	37.53	37.53	37.53	37.53	37.53	38.43
50	38.28	38.28	38.28	38.28	38.28	38.28	39.20
51	38.03	38.03	38.03	38.03	38.03	38.03	38.94
52	37.53	37.53	37.53	37.53	37.53	37.53	38.43
53	36.28	36.28	36.28	36.28	36.28	36.28	37.15
54	38.53	38.53	38.53	38.53	38.53	38.53	39.45
55	37.78	37.78	37.78	37.78	37.78	37.78	38.69
56	38.03	38.03	38.03	38.03	38.03	38.03	38.94
57	38.03	38.03	38.03	38.03	38.03	38.03	38.94
58	38.03	38.03	38.03	38.03	38.03	38.03	38.94
59	38.28	38.28	38.28	38.28	38.28	38.28	39.20
60	38.03	38.03	38.03	38.03	38.03	38.03	38.94
61	38.03	38.03	38.03	38.03	38.03	38.03	38.94
62	38.03	38.03	38.03	38.03	38.03	38.03	38.94
63	36.28	36.28	36.28	36.28	36.28	36.28	37.15
64	38.03	38.03	38.03	38.03	38.03	38.03	38.94
65	34.53	34.53	34.53	34.53	34.53	34.53	35.36
66	36.03	36.03	36.03	36.03	36.03	36.03	36.89
67	37.78	37.78	37.78	37.78	37.78	37.78	38.69
68	36.03	36.03	36.03	36.03	36.03	36.03	36.89
69	36.03	36.03	36.03	36.03	36.03	36.03	36.89
70	36.03	36.03	36.03	36.03	36.03	36.03	36.89

71	44.53	44.53	44.53	44.53	44.53	44.53	45.60
72	36.03	36.03	36.03	36.03	36.03	36.03	36.89
73	36.28	36.28	36.28	36.28	36.28	36.28	37.15
74	36.03	36.03	36.03	36.03	36.03	36.03	36.89
75	38.28	38.28	38.28	38.28	38.28	38.28	39.20
76	38.03	38.03	38.03	38.03	38.03	38.03	38.94
77	38.03	38.03	38.03	38.03	38.03	38.03	38.94
78	37.78	37.78	37.78	37.78	37.78	37.78	38.69
79	35.78	35.78	35.78	35.78	35.78	35.78	36.64
80	37.78	37.78	37.78	37.78	37.78	37.78	38.69
81	38.03	38.03	38.03	38.03	38.03	38.03	38.94
82	37.03	37.03	37.03	37.03	37.03	37.03	37.92
83	37.53	37.53	37.53	37.53	37.53	37.53	38.43
84	38.28	38.28	38.28	38.28	38.28	38.28	39.20
85	38.03	38.03	38.03	38.03	38.03	38.03	38.94
86	38.03	38.03	38.03	38.03	38.03	38.03	38.94
87	38.03	38.03	38.03	38.03	38.03	38.03	38.94
88	34.53	34.53	34.53	34.53	34.53	34.53	35.36
89	40.28	40.28	40.28	40.28	40.28	40.28	41.25
90	40.28	40.28	40.28	40.28	40.28	40.28	41.25
91	42.53	42.53	42.53	42.53	42.53	42.53	43.55
92	40.03	40.03	40.03	40.03	40.03	40.03	40.99
93	38.28	38.28	38.28	38.28	38.28	38.28	39.20
94	42.53	42.53	42.53	42.53	42.53	42.53	43.55
95	40.53	40.53	40.53	40.53	40.53	40.53	41.50
96	42.53	42.53	42.53	42.53	42.53	42.53	43.55
97	41.78	41.78	41.78	41.78	41.78	41.78	42.78
98	44.03	44.03	44.03	44.03	44.03	44.03	45.09
99	42.78	42.78	42.78	42.78	42.78	42.78	43.81
100	44.28	44.28	44.28	44.28	44.28	44.28	45.34
101	44.28	44.28	44.28	44.28	44.28	44.28	45.34
102	42.03	42.03	42.03	42.03	42.03	42.03	43.04
103	44.28	44.28	44.28	44.28	44.28	44.28	45.34
104	34.03	34.03	34.03	34.03	34.03	34.03	34.85
105	44.78	44.78	44.78	44.78	44.78	44.78	45.85
106	42.03	42.03	42.03	42.03	42.03	42.03	43.04
107	44.28	44.28	44.28	44.28	44.28	44.28	45.34
108	42.28	42.28	42.28	42.28	42.28	42.28	43.29
109	44.53	44.53	44.53	44.53	44.53	44.53	45.60
110	44.28	44.28	44.28	44.28	44.28	44.28	45.34
111	41.78	41.78	41.78	41.78	41.78	41.78	42.78
112	42.78	42.78	42.78	42.78	42.78	42.78	43.81
113	44.03	44.03	44.03	44.03	44.03	44.03	45.09
114	44.28	44.28	44.28	44.28	44.28	44.28	45.34

115	44.53	44.53	44.53	44.53	44.53	44.53	45.60
116	40.03	40.03	40.03	40.03	40.03	40.03	40.99
117	42.03	42.03	42.03	42.03	42.03	42.03	43.04
118	44.53	44.53	44.53	44.53	44.53	44.53	45.60
119	40.28	40.28	40.28	40.28	40.28	40.28	41.25
120	42.78	42.78	42.78	42.78	42.78	42.78	43.81
121	40.28	40.28	40.28	40.28	40.28	40.28	41.25
122	40.53	40.53	40.53	40.53	40.53	40.53	41.50
123	38.53	38.53	38.53	38.53	38.53	38.53	39.45
124	37.78	37.78	37.78	37.78	37.78	37.78	38.69
125	42.03	42.03	42.03	42.03	42.03	42.03	43.04
126	42.28	42.28	42.28	42.28	42.28	42.28	43.29
127	40.03	40.03	40.03	40.03	40.03	40.03	40.99
128	42.03	42.03	42.03	42.03	42.03	42.03	43.04
129	42.03	42.03	42.03	42.03	42.03	42.03	43.04
130	42.53	42.53	42.53	42.53	42.53	42.53	43.55
131	41.78	41.78	41.78	41.78	41.78	41.78	42.78
132	42.53	42.53	42.53	42.53	42.53	42.53	43.55
133	42.78	42.78	42.78	42.78	42.78	42.78	43.81
134	42.53	42.53	42.53	42.53	42.53	42.53	43.55
135	42.53	42.53	42.53	42.53	42.53	42.53	43.55
136	42.28	42.28	42.28	42.28	42.28	42.28	43.29
137	42.28	42.28	42.28	42.28	42.28	42.28	43.29
138	42.28	42.28	42.28	42.28	42.28	42.28	43.29
139	42.03	42.03	42.03	42.03	42.03	42.03	43.04
140	42.03	42.03	42.03	42.03	42.03	42.03	43.04
141	43.03	43.03	43.03	43.03	43.03	43.03	44.06
142	42.28	42.28	42.28	42.28	42.28	42.28	43.29
143	42.78	42.78	42.78	42.78	42.78	42.78	43.81
144	42.28	42.28	42.28	42.28	42.28	42.28	43.29
145	42.03	42.03	42.03	42.03	42.03	42.03	43.04
146	42.28	42.28	42.28	42.28	42.28	42.28	43.29
147	42.53	42.53	42.53	42.53	42.53	42.53	43.55
148	42.53	42.53	42.53	42.53	42.53	42.53	43.55
149	44.28	44.28	44.28	44.28	44.28	44.28	45.34
150	42.53	42.53	42.53	42.53	42.53	42.53	43.55
151	37.78	37.78	37.78	37.78	37.78	37.78	38.69
152	40.78	40.78	40.78	40.78	40.78	40.78	41.76
153	42.03	42.03	42.03	42.03	42.03	42.03	43.04
154	40.03	40.03	40.03	40.03	40.03	40.03	40.99
155	42.28	42.28	42.28	42.28	42.28	42.28	43.29
156	42.03	42.03	42.03	42.03	42.03	42.03	43.04
157	38.03	38.03	38.03	38.03	38.03	38.03	38.94
158	43.03	43.03	43.03	43.03	43.03	43.03	44.06

159	42. 28	42. 28	42. 28	42. 28	42. 28	42. 28	43. 29
160	42. 28	42. 28	42. 28	42. 28	42. 28	42. 28	43. 29
161	44. 03	44. 03	44. 03	44. 03	44. 03	44. 03	45. 09
162	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
163	42. 28	42. 28	42. 28	42. 28	42. 28	42. 28	43. 29
164	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
165	44. 03	44. 03	44. 03	44. 03	44. 03	44. 03	45. 09
166	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
167	34. 28	34. 28	34. 28	34. 28	34. 28	34. 28	35. 10
168	43. 78	43. 78	43. 78	43. 78	43. 78	43. 78	44. 83
169	42. 03	42. 03	42. 03	42. 03	42. 03	42. 03	43. 04
170	44. 53	44. 53	44. 53	44. 53	44. 53	44. 53	45. 60
171	43. 78	43. 78	43. 78	43. 78	43. 78	43. 78	44. 83
172	42. 28	42. 28	42. 28	42. 28	42. 28	42. 28	43. 29
173	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
174	42. 28	42. 28	42. 28	42. 28	42. 28	42. 28	43. 29
175	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
176	42. 28	42. 28	42. 28	42. 28	42. 28	42. 28	43. 29
177	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
178	42. 03	42. 03	42. 03	42. 03	42. 03	42. 03	43. 04
179	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
180	44. 53	44. 53	44. 53	44. 53	44. 53	44. 53	45. 60
181	44. 53	44. 53	44. 53	44. 53	44. 53	44. 53	45. 60
182	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
183	42. 28	42. 28	42. 28	42. 28	42. 28	42. 28	43. 29
184	40. 28	40. 28	40. 28	40. 28	40. 28	40. 28	41. 25
185	41. 78	41. 78	41. 78	41. 78	41. 78	41. 78	42. 78
186	38. 53	38. 53	38. 53	38. 53	38. 53	38. 53	39. 45
187	40. 28	40. 28	40. 28	40. 28	40. 28	40. 28	41. 25
188	42. 53	42. 53	42. 53	42. 53	42. 53	42. 53	43. 55
189	40. 53	40. 53	40. 53	40. 53	40. 53	40. 53	41. 50
190	42. 03	42. 03	42. 03	42. 03	42. 03	42. 03	43. 04
191	43. 03	43. 03	43. 03	43. 03	43. 03	43. 03	44. 06
192	44. 28	44. 28	44. 28	44. 28	44. 28	44. 28	45. 34
193	44. 78	44. 78	44. 78	44. 78	44. 78	44. 78	45. 85
194	42. 78	42. 78	42. 78	42. 78	42. 78	42. 78	43. 81
195	28. 03	28. 03	28. 03	28. 03	28. 03	28. 03	28. 70
196	34. 78	34. 78	34. 78	34. 78	34. 78	34. 78	35. 61
197	34. 78	34. 78	34. 78	34. 78	34. 78	34. 78	35. 61
198	34. 53	34. 53	34. 53	34. 53	34. 53	34. 53	35. 36
199	34. 53	34. 53	34. 53	34. 53	34. 53	34. 53	35. 36
200	34. 78	34. 78	34. 78	34. 78	34. 78	34. 78	35. 61

